CYBERSECURITY HOME LAB REPORT

File Integrity Monitoring with Wazuh

Detecting file creation, modification and deletion on Windows 10, investigating an unexpected change, and understanding the limits of FIM

Day 9 — Wazuh SOC Analyst Home Lab, 14-Day Intensive Program

Author	Aigbokhaode Hope Imomoh
Role	SOC Analyst Trainee
Date	11 November 2026
Version	1.0
Environment	Oracle VirtualBox: Windows 10 endpoint (Wazuh agent, Sysmon) and Ubuntu server (Wazuh, Splunk Enterprise 9.4.2)
Evidence period	Screenshots captured on 11 November 2026, 13:29 to 15:28

Evidence statement

All results come from an isolated home lab. Screenshots in this report are cropped for presentation only; the originals are retained unchanged. Evidence gaps and unresolved items are stated openly in Section 16.3 and Chapter 10.

Table of Contents

1. Executive Summary 6
2. Introduction and Objectives 7
2.1 Background and motivation 7
2.2 The lab brief 7
2.3 Objectives 7
2.4 Scope 8
2.5 Measurable success criteria 8
3. Technical Background 9
3.1 File Integrity Monitoring (FIM) 9
3.2 Hashes 9
3.3 Scheduled, real-time and who-data modes 9
3.4 From event to dashboard: six separate stages 9
3.5 Sysmon Event ID 1 and MITRE ATT&CK 10
4. Lab Environment and Architecture 11
4.1 Components 11
4.2 Architecture and data flow 11
5. Prerequisites and Safety 12
5.1 Prerequisites 12
5.2 Safety measures used 12
5.3 Restoring the lab if something goes wrong 12
6. Step-by-Step Implementation 13
6.1 Step 1 — Create the safe test directory 13
6.2 Step 2 — Back up and edit the agent configuration 13
6.3 Step 3 — Restart the agent and check the service 14
6.4 Step 4 — Verify that FIM started (agent log) 15
6.5 Step 5 — Test 1: create a file 15
6.6 Step 6 — Test 2: modify the file 17
6.7 Step 7 — Test 3: delete the file 19
6.8 Step 8 — Test 4: the unexpected modification 20
6.9 Step 9 — Correlate with Sysmon in Splunk 24
7. Configuration and Detection Logic 26
7.1 The configuration change (the only one made) 26
7.2 Original, changed and recommended 26
7.3 Fields used for detection and investigation 26
7.4 Queries used 27
7.5 Example improvement (NOT tested in this lab) 27
8. Testing and Results 28
8.1 Test scenarios 28
8.2 File timeline (required deliverable) 28
8.3 What succeeded, failed and remains untested 29
9. Evidence and Screenshot Analysis 30
9.1 Evidence handling 30
9.2 Background alerts seen alongside the test 30
9.3 What each evidence group proves 30
10. Troubleshooting and Root-Cause Analysis 32
11. Security Findings and Risk Analysis 33
11.1 Findings 33
11.2 Analyst notes — the six analyst questions 33
11.3 Why context matters 34
12. Mitigation and Remediation 35
12.1 Implemented in the lab 35
12.2 Recommendations (not implemented) 35

13. What I Learned **37**
14. Practical Knowledge and Competencies **38**
14.1 Suggested next exercises 38
15. Reproduction Guide **39**
15.1 Initial state 39
15.2 Setup 39
15.3 Test activities 39
15.4 Verification and evidence collection 40
15.5 Troubleshooting 40
15.6 Cleanup and rollback (documented, not executed in this lab) 40
16. Limitations and Future Improvements **41**
16.1 Limits of the experiment 41
16.2 Timestamp and display anomalies 41
16.3 Evidence gaps 41
16.4 False positives and false negatives 41
16.5 Future improvements 41
17. Conclusion **42**
A. Appendix A — Command History **43**
B. Appendix B — Configuration and Log Excerpts **44**
C. Appendix C — Evidence Register **45**
D. Appendix D — Glossary **47**
E. Appendix E — References **48**

List of Figures

Figure 1 — Day 9 lab brief (source: programme PDF, page 11 and following), reproduced as the reference for this report's structure..... 7
Figure 2 — The six stages between a file change and an analyst reading the alert..... 10
Figure 3 — Laboratory architecture and data flow..... 11
Figure 4 — Creating and confirming the C:\FIM_Test directory..... 13
Figure 5 — Edited <syscheck> block before saving; the stray "s" is visible after </skip_nfs> at the top..... 14
Figure 6 — After correction: </skip_nfs> ends cleanly and the new C:\FIM_Test line sits under <frequency>..... 14
Figure 7 — Backup, edit, restart and service status..... 14
Figure 8 — Agent log confirming that c:\fim_test is monitored in real-time with who-data..... 15
Figure 9 — Creation command and its Get-Date output (cropped from a longer session screenshot)..... 15
Figure 10 — Events list: c:\fim_test\payroll.txt — added — rule 554 — at the top of the list..... 15
Figure 11 — Creation alert: process powershell.exe (PID 6324), user Administrator, event added, mode whodata. 16
Figure 12 — Creation alert: path, SHA-1, SHA-256, size 52 bytes, owner group Administrators..... 16
Figure 13 — Modification command and time (2:36:24 PM)..... 17
Figure 14 — Events list: a "modified" row (rule 550, level 7) above the earlier "added" row..... 17
Figure 15 — The alert's full_log: size changed from 52 to 96, with before/after checksums..... 17
Figure 16 — Who-data and diff: same PID 6324, changed attributes size, mtime, md5, sha1, sha256..... 18
Figure 17 — Before/after hashes and sizes for the modification..... 18
Figure 18 — Deletion command and time (2:49:25 PM)..... 19
Figure 19 — Three file events for payroll.txt now in the list: added, modified, deleted..... 19
Figure 20 — Deletion alert: decoder syscheck_deleted, mode whodata..... 20
Figure 21 — Deletion alert: user Administrator, powershell.exe PID 6324, last-known MD5 5f9938...17f4c and SHA-1 f0e4ad...f7d1..... 20
Figure 22 — Creating config.ini from PowerShell (3:05:37 PM)..... 21
Figure 23 — The appended setting typed into an Administrator Command Prompt (cropped to the window; the original shows 3:07 PM in the taskbar)..... 21
Figure 24 — Events list showing config.ini — added (15:05:38) and modified (15:07:00)..... 21
Figure 25 — Modification alert: process C:\Windows\System32\cmd.exe PID 6648, user Administrator, diff > mode=debug_open..... 22
Figure 26 — Modification alert: before/after hashes for config.ini..... 22
Figure 27 — Creation alert for config.ini — rule 554, audit process ID 6324 (the PowerShell session)..... 23
Figure 28 — Creation alert: SHA-1, SHA-256 and size (60 bytes) of the baseline file..... 23
Figure 29 — Splunk: three cmd.exe process-creation events on DESKTOP-T6T1N2V..... 24
Figure 30 — Scheduled-mode registry alerts (04:06–04:07) listed beside the test events; they are background noise, not part of the test..... 30

List of Tables

Table 1 — Summary of verified outcomes..... 6
Table 2 — Success criteria and result..... 8
Table 3 — FIM monitoring modes..... 9
Table 4 — Pipeline stages and where to check each..... 10
Table 5 — Laboratory components (as evidenced by the screenshots)..... 11
Table 6 — Evidence captured for the creation event..... 16
Table 7 — Evidence captured for the modification event..... 18
Table 8 — Evidence for config.ini..... 23
Table 9 — Interpretation of the three cmd.exe events..... 24
Table 10 — Line-by-line explanation..... 26
Table 11 — Configuration status..... 26
Table 12 — Wazuh FIM fields used in this lab..... 26
Table 13 — Searches used or shown in the evidence..... 27
Table 14 — Test scenarios, expectations and outcomes..... 28
Table 15 — File timeline..... 28
Table 16 — Evidence assessment..... 30
Table 17 — Problems encountered..... 32
Table 18 — Findings and confidence..... 33

Table 19 — Controls actually implemented....35
Table 20 — Skills mapped to lab tasks....38
Table 21 — Commands to run and what to expect....39
Table 22 — Common problems....40
Table 23 — Commands executed....43
Table 24 — Alert IDs....44
Table 25 — Evidence register....45
Table 26 — Glossary....47

Abbreviations and Acronyms

Table 27 — Abbreviations

Abbreviation	Meaning
ACL	Access control list
ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
DQL	Dashboard query language
FIM	File integrity monitoring
PID	Process ID
SID	Security identifier
SIEM	Security information and event management
SOC	Security operations centre
SPL	Search Processing Language (Splunk)
VM	Virtual machine

1. Executive Summary

This report documents **Day 9 — File Integrity Monitoring** of a 14-day Wazuh SOC Analyst home-lab programme. The goal was to use Wazuh File Integrity Monitoring (FIM) to detect controlled file creation, modification and deletion on a Windows 10 endpoint, record the evidence an analyst needs (path, user, process, time and hashes), investigate an unexpected modification, and explain the limits of FIM.

The security problem

Attackers and careless administrators alike change files: configuration files are weakened, scripts are replaced, data is tampered with. A Security Operations Centre (SOC) needs to know **that** a file changed, **who** changed it, **with which program**, and **whether the change is expected**. FIM answers the first question automatically; the analyst must supply the rest through investigation.

What was done

- A dedicated test folder, `C:\FIM_Test`, was created on the Windows 10 VM and added to the Wazuh agent configuration with real-time monitoring, who-data (user/process attribution) and content-diff reporting.
- The agent log confirmed that FIM and the who-data engine had started (log messages 6003, 6012 and 6019).
- `payroll.txt` was created, modified and deleted from PowerShell; each action produced a Wazuh alert within about one to two seconds.
- `config.ini` was created from PowerShell and then changed from a **different** program, `cmd.exe`, to simulate an unexpected, security-weakening modification (staged by the author in an isolated lab; not a real incident).
- The `cmd.exe` activity was correlated in Splunk using Sysmon process-creation events (Event ID 1), which showed the process was started interactively from `explorer.exe`.

Key verified results

Table 1 — Summary of verified outcomes

Area	Outcome	Status
FIM configuration	Directory added with `realtime`, `whodata`, `report_changes`; agent log confirms real-time and who-data engines started.	Verified
File lifecycle detection	Add (rule 554), modify (rule 550) and delete (description "File deleted.") alerts were generated for `payroll.txt`.	Verified; rule ID of the delete alert not captured on screen
Attribution	Each alert named the user (`Administrator`) and the process (`powershell.exe` PID 6324, or `cmd.exe` PID 6648).	Verified
Hash evidence	MD5, SHA-1 and SHA-256 before/after values were captured for modifications.	Verified
Unexpected change	`cmd.exe` appended `mode=debug_open` to `config.ini`; the diff and Sysmon parent process were captured.	Verified (simulated by the author)
FIM limitations	Observed: FIM did not show the typed command; background registry alerts create noise; Wazuh's own helper `cmd.exe` can resemble suspicious activity.	Observed and analysed

2. Introduction and Objectives

2.1 Background and motivation

This laboratory is part of a hands-on programme for becoming an entry-level SOC analyst. Earlier days built the home lab (Wazuh SIEM, Splunk, Sysmon and a Windows 10 workstation). Day 9 adds a detection capability that is used in real SOCs and in compliance frameworks such as PCI DSS requirement 11.5: knowing when important files change.

2.2 The lab brief

The programme brief for Day 9 defines the tasks, deliverables and analyst questions used to structure this report (Figure 1).

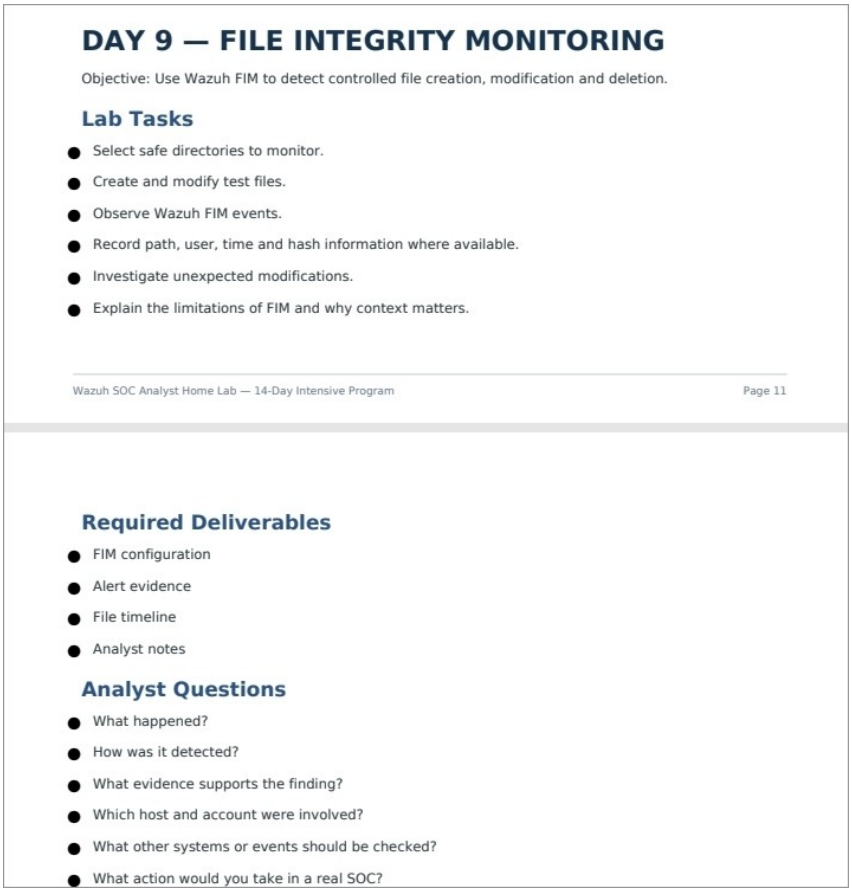


Figure 1 — Day 9 lab brief (source: programme PDF, page 11 and following), reproduced as the reference for this report's structure.

2.3 Objectives

1. Select safe directories to monitor.
2. Create and modify test files.
3. Observe Wazuh FIM events.
4. Record path, user, time and hash information where available.
5. Investigate unexpected modifications.
6. Explain the limitations of FIM and why context matters.

2.4 Scope

In scope: one Windows 10 endpoint (agent ID 006), one monitored folder, the Wazuh manager/dashboard and Splunk on the Ubuntu VM. Out of scope: monitoring of system directories, active response, custom rules, and alert forwarding to external systems. Nothing outside the isolated VirtualBox network was touched.

2.5 Measurable success criteria

Table 2 — Success criteria and result

ID	Criterion	Result
S1	FIM is enabled for the test folder and the agent log confirms it.	Met (Figures 5–8)
S2	A file creation produces an alert with path, user, process and hashes.	Met
S3	A file modification produces an alert with before/after hashes and a content diff.	Met
S4	A file deletion produces an alert with user, process and last-known hashes.	Met (rule ID not captured)
S5	An unexpected change from a different process is detected and traced to its process.	Met
S6	The change is correlated with a second data source.	Met (Sysmon via Splunk)
S7	The limits of FIM are explained using observations from the lab.	Met (Chapters 11 and 16)

3. Technical Background

This chapter explains the ideas used in the lab in plain language first, then technically.

3.1 File Integrity Monitoring (FIM)

Plain language: FIM is a security camera for files. It remembers what a file looked like, notices when that picture changes, and raises an alert.

Technical: the Wazuh agent module **syscheck** records attributes of monitored files (size, permissions, owner, group, modification time, inode and cryptographic hashes). When a later scan or real-time notification differs from the stored baseline, syscheck sends a FIM event to the manager.

3.2 Hashes

A hash (MD5, SHA-1, SHA-256) is a fixed-length fingerprint of a file's contents. Changing even one character changes the hash completely. FIM compares the "before" and "after" hashes, so a different hash is proof that the content changed, even if the size or timestamp is unchanged. SHA-256 is preferred for evidence; MD5 and SHA-1 are collision-weak but still useful for matching.

3.3 Scheduled, real-time and who-data modes

Table 3 — FIM monitoring modes

Mode	How it works	Seen in this lab
Scheduled	Full scan at an interval (the default `frequency` here is 43200 seconds = 12 hours) or on start.	Registry and system-path alerts with `syscheck.mode: scheduled`.
Real-time (`realtime="yes"`)	The operating system notifies the agent as soon as a change happens.	Alerts arrived within about 1–2 seconds.
Who-data (`whodata="yes"`)	Uses Windows auditing to add the **user** and **process** that made the change.	`syscheck.audit.user.name`, `syscheck.audit.process.name`, PID.
Report changes (`report_changes="yes"`)	Stores a copy of text files so a content **diff** can be shown.	`syscheck.diff` showed the added line.

3.4 From event to dashboard: six separate stages

Beginners often treat "a log appeared" as one step. It is at least six. Knowing which stage failed is the key to troubleshooting.

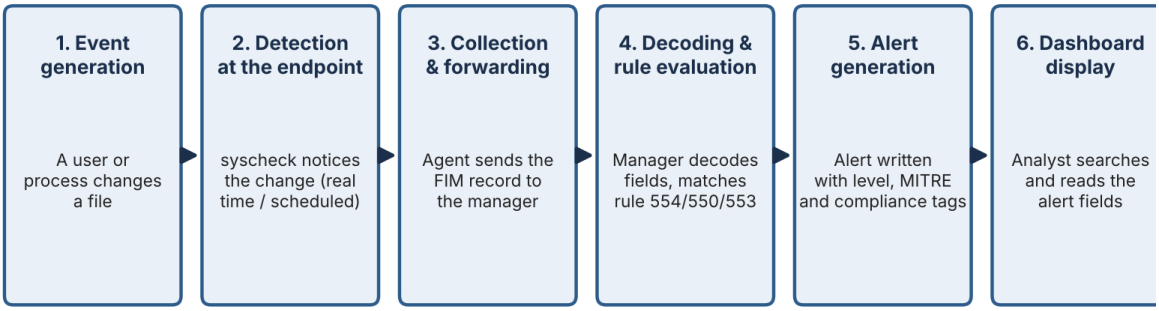


Figure 2 — The six stages between a file change and an analyst reading the alert.

Table 4 — Pipeline stages and where to check each

Stage	Meaning	Where to verify in this lab
1 Event generation	The change really happens.	The PowerShell/cmd command and its output.
2 Endpoint detection	syscheck notices it.	`ossec.log` messages 6003, 6012, 6019.
3 Collection/forwarding	The agent delivers the record.	Agent service running; alert reaches the manager.
4 Decoding and rule evaluation	The manager splits fields and picks a rule.	`decoder.name`, `rule.id`.
5 Alert generation	A scored alert is stored.	`rule.level`, `rule.groups`, MITRE tags.
6 Dashboard display	The analyst finds it.	Wazuh dashboard Events view and filters.

3.5 Sysmon Event ID 1 and MITRE ATT&CK

Sysmon is a Microsoft Sysinternals service that logs detailed host activity. **Event ID 1** is "process created" and records the image path, the parent process and the command line. FIM says what changed; Sysmon helps show how the process that did it was started.

Wazuh tagged the modification alert with MITRE ATT&CK technique **T1565.001 (Stored Data Manipulation)** under the tactic Impact. Registry change alerts carried **T1112 (Modify Registry)**.

4. Lab Environment and Architecture

4.1 Components

Table 5 — Laboratory components (as evidenced by the screenshots)

Component	Details
Virtualisation	Oracle VirtualBox; guests shown as "Windows 10 [Running]" and an Ubuntu desktop.
Monitored endpoint	Windows 10, version 10.0.19045.6466; host name `DESKTOP-T6T1N2V`; IP `192.168.56.109`; Wazuh agent ID `006`; service `WazuhSvc` (display name "wazuh").
Wazuh server	Manager name "Ubuntu"; dashboard reached at `192.168.56.101` (Wazuh version not captured in the evidence).
Splunk	Splunk Enterprise 9.4.2 in the Ubuntu browser at `127.0.0.1:8000`; data in `index=sysmon`.
Endpoint telemetry	Sysmon Event ID 1 records visible in Splunk; account shown as `HOPELABS\Administrator`.
Network	Addresses in `192.168.56.0/24`, the VirtualBox host-only range; no external network required for the test.

4.2 Architecture and data flow

The agent on the Windows VM watches the test folder and sends FIM events to the manager on the Ubuntu VM, where decoders and rules turn them into alerts shown in the dashboard. Separately, Sysmon process-creation events were searchable in Splunk. How Sysmon data reaches Splunk was configured on an earlier day and is not part of this evidence, so it is drawn as a dashed line.

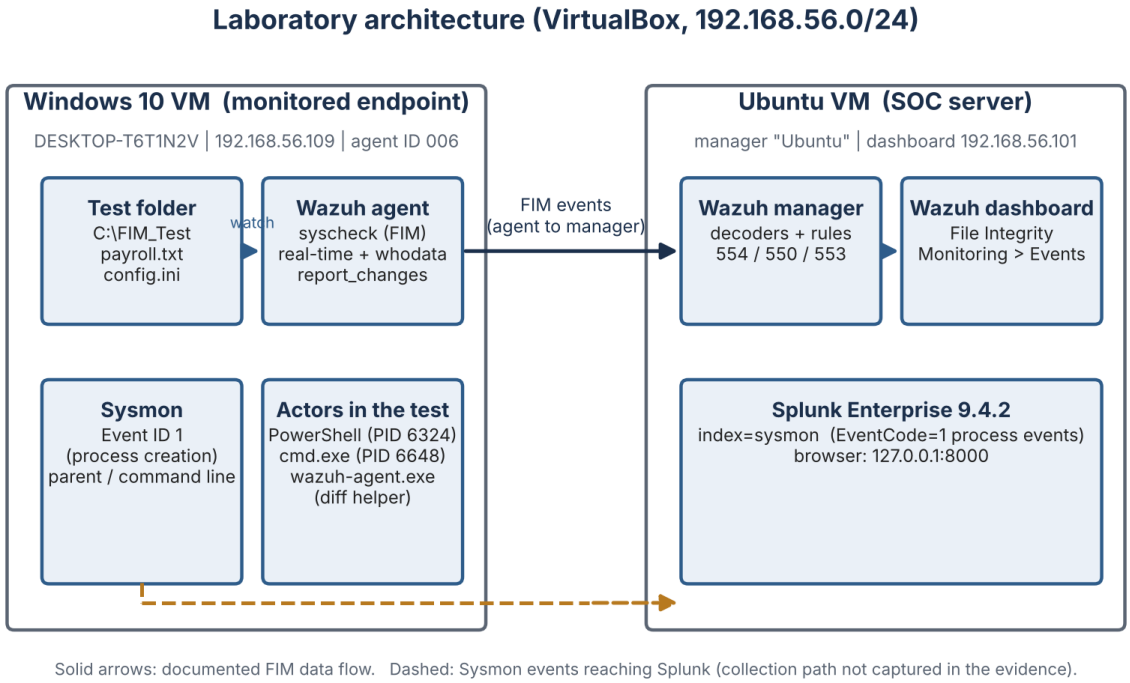


Figure 3 — Laboratory architecture and data flow.

5. Prerequisites and Safety

5.1 Prerequisites

- A working Wazuh manager, indexer and dashboard, with the Windows 10 machine enrolled as an agent and showing as active.
- Administrator rights on the Windows VM (needed to edit the agent configuration, restart the service and create audit entries for who-data).
- A web browser on the Ubuntu VM for the Wazuh dashboard and Splunk.
- Sysmon installed on the Windows VM and its events indexed in Splunk (needed only for the correlation step).

5.2 Safety measures used

- **Isolated scope:** only a new, empty folder (`C:\FIM_Test`) was monitored; no system or personal files were edited.
- **Backup before change:** `ossec.conf` was copied to `ossec.conf.bak` before editing.
- **Harmless content:** the test files contain dummy text; "payroll" is a name, not real data.
- **Snapshot recommended:** take a VirtualBox snapshot before configuration changes (recommended; no snapshot is shown in the evidence).

> **Warning**
> Never monitor a busy system directory with `report_changes` or real-time mode on a production machine without testing: it can produce heavy alert volumes and store copies of sensitive text files.

5.3 Restoring the lab if something goes wrong

Restore the backup and restart the agent (documented for the reader; this restore was not needed in the lab):

```
Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf.bak" "C:\Program Files
(x86)\ossec-agent\ossec.conf" -Force
Restart-Service -Name wazuh
```

6. Step-by-Step Implementation

All times below are the **Windows VM clock** as printed by `Get-Date` or shown in the taskbar, and the **Wazuh alert timestamp** as displayed by the dashboard. The two agreed to within about one to two seconds for every event (see Section 16.2 for one display anomaly). "PS>" means an **Administrator** PowerShell prompt on the Windows VM.

6.1 Step 1 — Create the safe test directory

What / where	Create an empty folder on the Windows VM. Administrator PowerShell.
Why	FIM should be tested on a folder that is safe to change and that nothing else uses, so every alert is caused by the lab action.
Command	`New-Item -ItemType Directory C:\FIM_Test` then `Test-Path C:\FIM_Test`
Meaning	`New-Item -ItemType Directory` creates a folder; `Test-Path` returns `True` if the path exists.
Expected	A directory listing for `FIM_Test`, then `True`.
Actual	Folder created at 1:29 PM; `Test-Path` returned `True` (Figure 4).
Common errors	"Access denied": PowerShell was not started as Administrator. "Already exists": the folder is left over from an earlier attempt; delete its contents first.

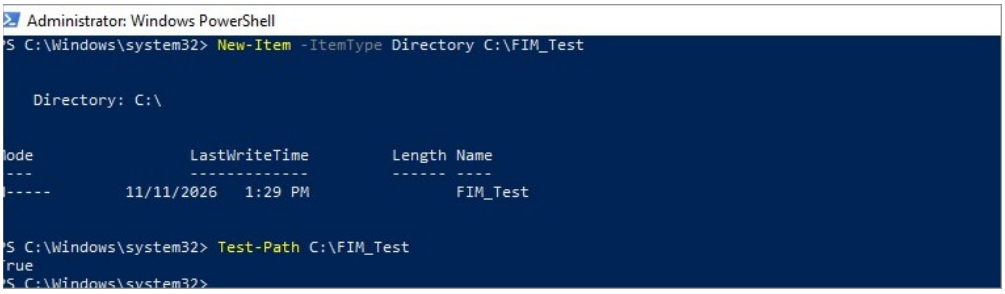


Figure 4 — Creating and confirming the C:\FIM_Test directory.

6.2 Step 2 — Back up and edit the agent configuration

What / where	Copy `ossec.conf`, then open it in Notepad. Windows VM, Administrator PowerShell.
Why	The agent only monitors what its configuration lists. A backup allows a clean rollback.
Commands	`Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" "C:\Program Files (x86)\ossec-agent\ossec.conf.bak"` `notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"`
Expected	A copy named `ossec.conf.bak`; Notepad opens the live configuration.
Actual	Both commands ran without error (visible in Figure 7).

In Notepad the `<syscheck>` block was found (File integrity monitoring comment). `<disabled>no</disabled>` confirmed FIM was already enabled. The following line was added directly under `<frequency>43200</frequency>`:

```
<directories check_all="yes" realtime="yes" whodata="yes" report_changes="yes">C:\FIM_Test</directories>
```

A formatting fault was noticed while checking the file. In the neighbouring `<sca>` block the line read `<skip_nfs>yes</skip_nfs>s`, with a stray letter **s** after the closing tag (visible at the top of Figure 5). It was deleted before saving (Figure 6). Whether the original text would have stopped the agent was not tested; removing it was a precaution, because malformed XML can prevent the agent from reading its configuration.

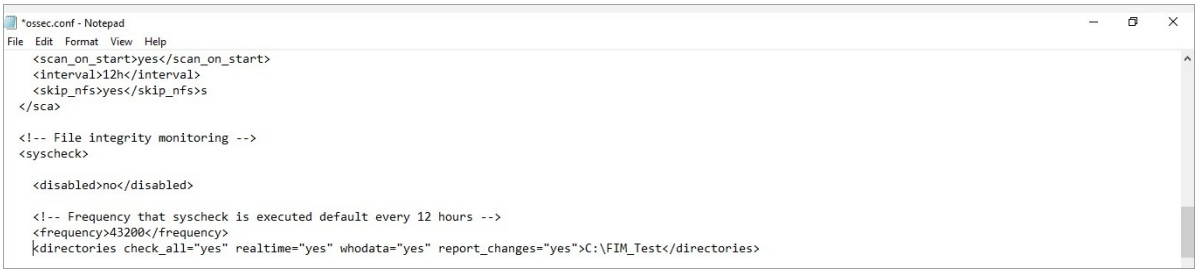


Figure 5 — Edited <syscheck> block before saving; the stray "s" is visible after </skip_nfs> at the top.



Figure 6 — After correction: </skip_nfs> ends cleanly and the new C:\FIM_Test line sits under <frequency>.

> **Note**
> Edit the file as Administrator and keep the XML tags exactly balanced. If the agent refuses to start after an edit, restore `ossec.conf.bak` (Section 5.3).

6.3 Step 3 — Restart the agent and check the service

Commands	`Restart-Service -Name wazuh` `Get-Service -Name wazuh`
Why	The agent reads `ossec.conf` only at start-up.
Expected	Status `Running`.
Actual	Service `WazuhSvc` (display name "wazuh") reported `Running` (Figure 7).
Common errors	Service name not found: use the Services app to confirm the name. Status `Stopped`: the configuration is probably malformed; restore the backup and repeat.



Figure 7 — Backup, edit, restart and service status.

6.4 Step 4 — Verify that FIM started (agent log)

Command	`Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.log" -Pattern "syscheck|FIM_Test|whodata|real-time" | Select-Object -Last 10`
Meaning	`Select-String` searches the log for any of the words (the `|` inside the quotes means "or"); `Select-Object -Last 10` keeps the final ten matches.
Expected	Messages showing the path is monitored and that real-time and who-data engines started.
Actual	Three messages (Figure 8): **6003** monitoring path `c:\fim_test` with options including `hash_md5`, `hash_sha1`, `hash_sha256`, `report_changes` and `whodata` (13:52:47); **6012** real-time FIM started (13:54:02); **6019** real-time who-data engine started (13:54:08).
Why it matters	This is pipeline stage 2 (Section 3.4). Without these lines the later tests could not be trusted to have been monitored.

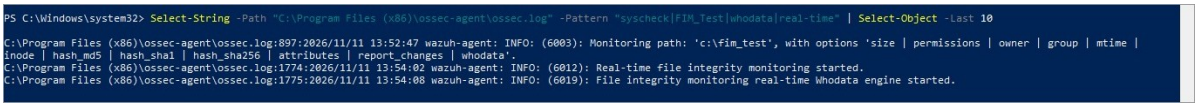


Figure 8 — Agent log confirming that c:\fim_test is monitored in real-time with who-data.

6.5 Step 5 — Test 1: create a file

Command	`Get-Date; "Employee salary list v1" | Out-File C:\FIM_Test\payroll.txt`
Meaning	`Get-Date` prints the time (used as the evidence anchor); the text is piped (`|`) into `Out-File`, which writes it to `payroll.txt`.
Expected	A rule 554 "File added to the system" alert within seconds, with who-data.
Actual	File created at **1:58:07 PM** (Figure 9). Alert timestamped **13:58:08**.

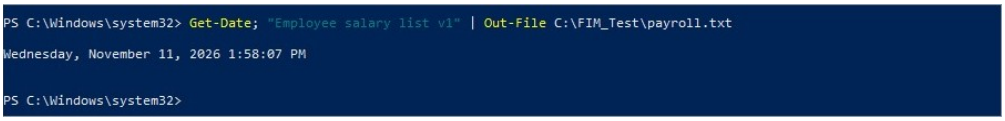


Figure 9 — Creation command and its Get-Date output (cropped from a longer session screenshot).

The dashboard Events view lists the new row at the top (Figure 10) next to unrelated registry alerts that are explained in Chapter 9. Expanding the alert shows the who-data fields (Figure 11) and the hashes (Figure 12).

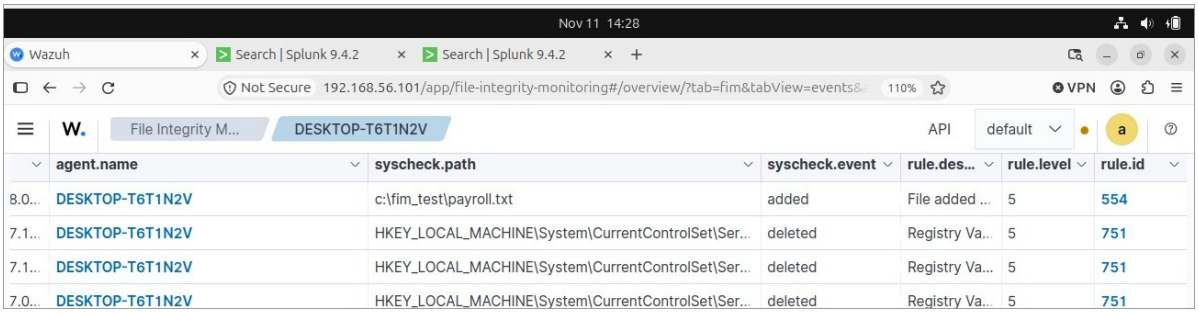
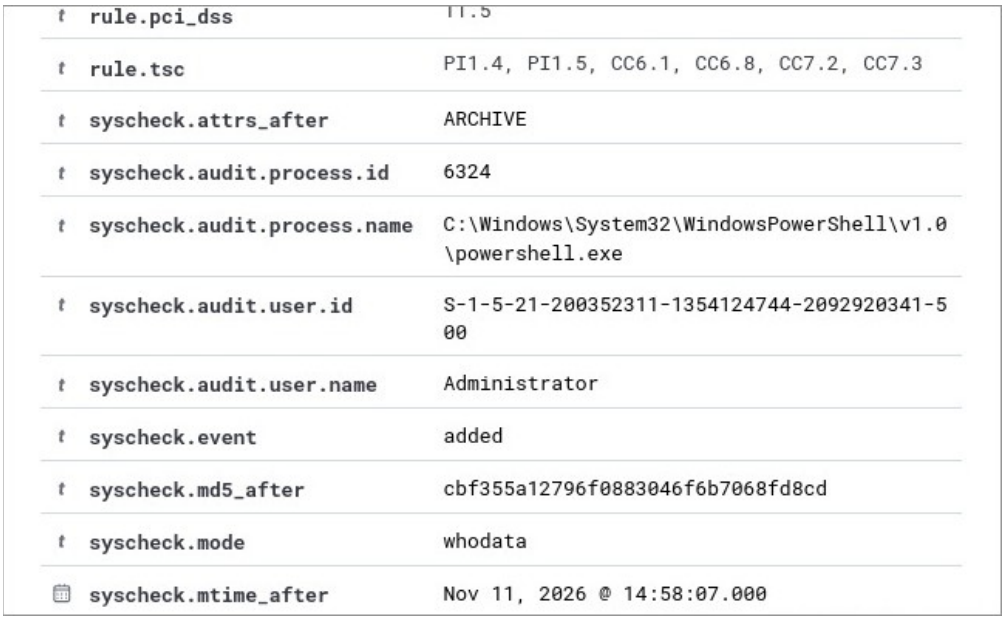
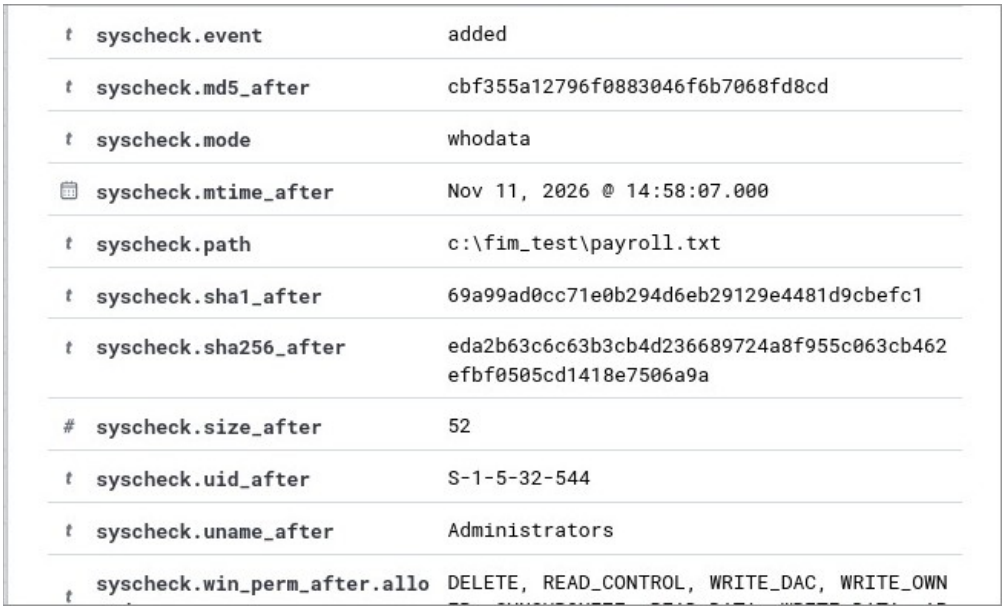


Figure 10 — Events list: c:\fim_test\payroll.txt — added — rule 554 — at the top of the list.



t rule.pci_dss	11.5
t rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
t syscheck.attrs_after	ARCHIVE
t syscheck.audit.process.id	6324
t syscheck.audit.process.name	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
t syscheck.audit.user.id	S-1-5-21-200352311-1354124744-2092920341-500
t syscheck.audit.user.name	Administrator
t syscheck.event	added
t syscheck.md5_after	cbf355a12796f0883046f6b7068fd8cd
t syscheck.mode	whodata
syscheck.mtime_after	Nov 11, 2026 @ 14:58:07.000

Figure 11 — Creation alert: process powershell.exe (PID 6324), user Administrator, event added, mode whodata.



t syscheck.event	added
t syscheck.md5_after	cbf355a12796f0883046f6b7068fd8cd
t syscheck.mode	whodata
syscheck.mtime_after	Nov 11, 2026 @ 14:58:07.000
t syscheck.path	c:\fim_test\payroll.txt
t syscheck.sha1_after	69a99ad0cc71e0b294d6eb29129e4481d9cbefc1
t syscheck.sha256_after	eda2b63c6c63b3cb4d236689724a8f955c063cb462efbf0505cd1418e7506a9a
# syscheck.size_after	52
t syscheck.uid_after	S-1-5-32-544
t syscheck.uname_after	Administrators
t syscheck.win_perm_after.allo	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWN

Figure 12 — Creation alert: path, SHA-1, SHA-256, size 52 bytes, owner group Administrators.

Table 6 — Evidence captured for the creation event

Field	Value
Alert time / rule	13:58:08 — rule **554**, level 5, "File added to the system." (group `syscheck_entry_added`)
Path	`c:\fim_test\payroll.txt`
User / process	`Administrator` (SID ends `-500`) / `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, PID 6324
Size	52 bytes
MD5	`cbf355a12796f0883046f6b7068fd8cd`
SHA-1	`69a99ad0cc71e0b294d6eb29129e4481d9cbefc1`
SHA-256	`eda2b63c6c63b3cb4d236689724a8f955c063cb462efbf0505cd1418e7506a9a`

Field	Value
Alert ID	`1794401888.1058407`

6.6 Step 6 — Test 2: modify the file

Command	`Get-Date; Add-Content C:\FIM_Test\payroll.txt "Salary changed by me"`
Meaning	`Add-Content` appends text to the end of a file.
Expected	A rule 550 alert with before/after hashes and a diff.
Actual	Command ran at **2:36:24 PM** (Figure 13); alert at **14:36:25** (Figure 14).

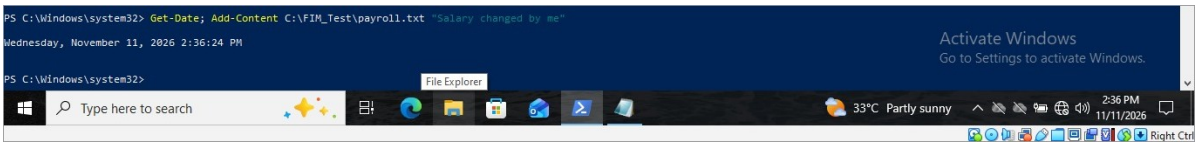


Figure 13 — Modification command and time (2:36:24 PM).

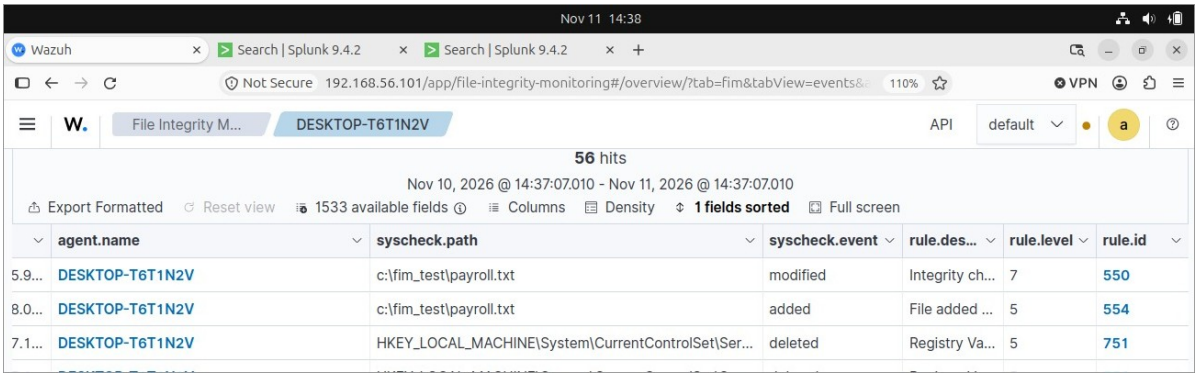
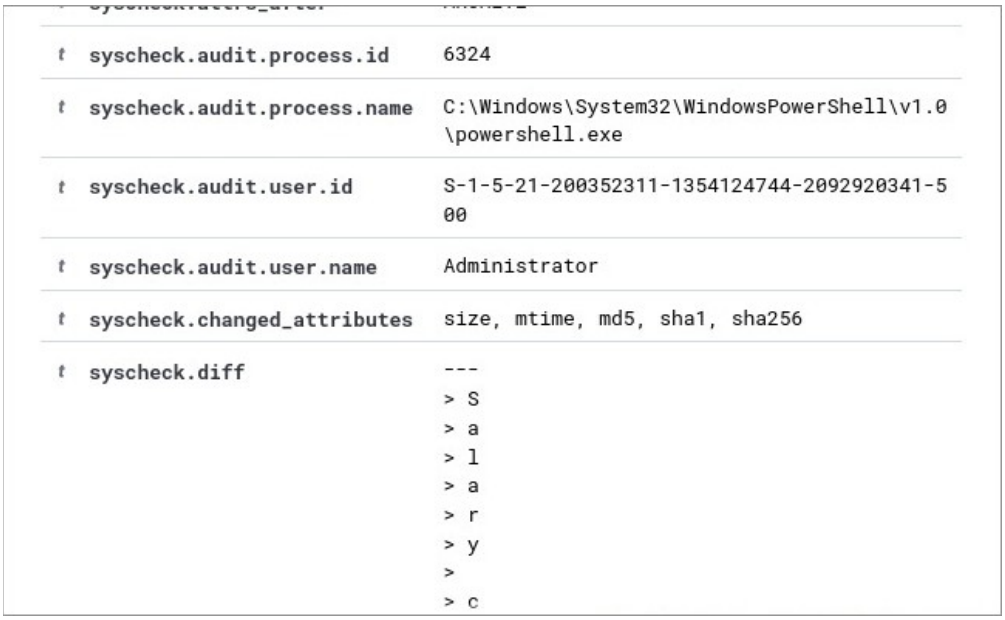


Figure 14 — Events list: a "modified" row (rule 550, level 7) above the earlier "added" row.

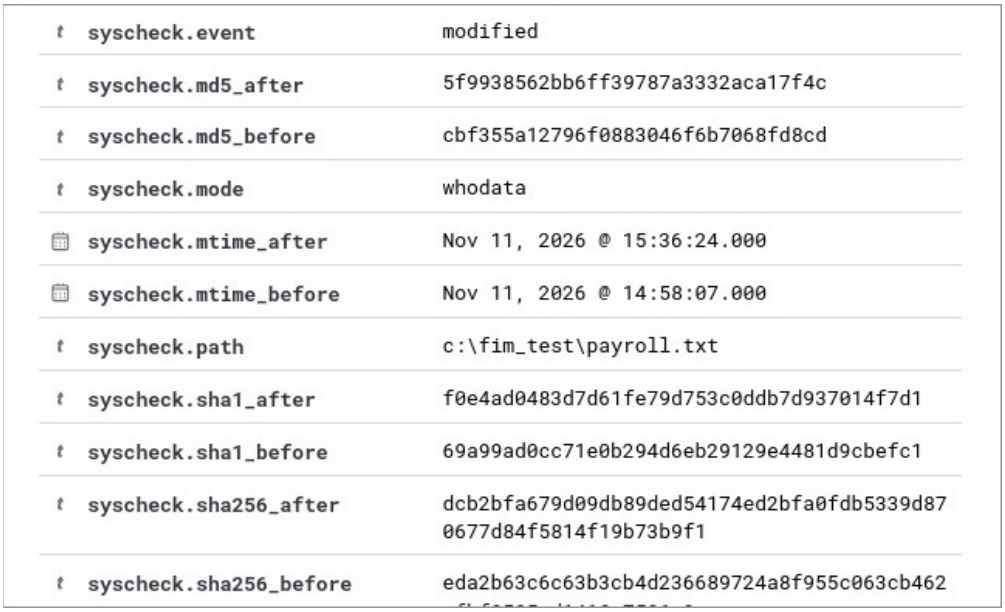


Figure 15 — The alert's full_log: size changed from 52 to 96, with before/after checksums.



t syscheck.audit.process.id	6324
t syscheck.audit.process.name	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
t syscheck.audit.user.id	S-1-5-21-200352311-1354124744-2092920341-500
t syscheck.audit.user.name	Administrator
t syscheck.changed_attributes	size, mtime, md5, sha1, sha256
t syscheck.diff	--- > S > a > l > a > r > y > > c

Figure 16 — Who-data and diff: same PID 6324, changed attributes size, mtime, md5, sha1, sha256.



t syscheck.event	modified
t syscheck.md5_after	5f9938562bb6ff39787a3332aca17f4c
t syscheck.md5_before	cbf355a12796f0883046f6b7068fd8cd
t syscheck.mode	whodata
syscheck.mtime_after	Nov 11, 2026 @ 15:36:24.000
syscheck.mtime_before	Nov 11, 2026 @ 14:58:07.000
t syscheck.path	c:\fim_test\payroll.txt
t syscheck.sha1_after	f0e4ad0483d7d61fe79d753c0ddb7d937014f7d1
t syscheck.sha1_before	69a99ad0cc71e0b294d6eb29129e4481d9cbefc1
t syscheck.sha256_after	dcb2bfa679d09db89ded54174ed2bfa0fdb5339d870677d84f5814f19b73b9f1
t syscheck.sha256_before	eda2b63c6c63b3cb4d236689724a8f955c063cb462

Figure 17 — Before/after hashes and sizes for the modification.

Table 7 — Evidence captured for the modification event

Field	Before	After
Size	52	96
MD5	`cbf355a12796f0883046f6b7068fd8cd`	`5f9938562bb6ff39787a3332aca17f4c`
SHA-1	`69a99ad0cc71e0b294d6eb29129e4481d9cbefc1`	`f0e4ad0483d7d61fe79d753c0ddb7d937014f7d1`
SHA-256	`eda2b63c6c63b3cb4d236689724a8f955c063cb462efbf0505cd1418e7506a9a`	`dcb2bfa679d09db89ded54174ed2bfa0fdb5339d870677d84f5814f19b73b9f1`
Rule	Rule **550**, level 7, "Integrity checksum changed.", MITRE T1565.001	
User / process	`Administrator` / `powershell.exe` PID 6324	

> **Evidence**
> The "before" hashes on the modification alert equal the "after" hashes on the creation alert. FIM therefore preserved an unbroken chain of file states.

Reader's note on sizes (inference, not shown on screen): 52 bytes for a 23-character line, and 44 bytes added for a 20-character line, are consistent with Windows PowerShell's default UTF-16 text output (two bytes per character plus a byte-order mark and a line ending). The diff in Figure 16 also listed the new text one character per row, which fits that encoding.

6.7 Step 7 — Test 3: delete the file

Command	`Get-Date; Remove-Item C:\FIM_Test\payroll.txt`
Expected	A "file deleted" alert with the last-known hashes.
Actual	Command ran at **2:49:25 PM** (Figure 18); alert at **14:49:26** (Figure 19). The alert's `full_log` reads "File 'c:\fim_test\payroll.txt' deleted, Mode: whodata" (Figure 20); who-data named `Administrator` / `powershell.exe` PID 6324 (Figure 21).
Gap in evidence	The screenshots show the description "File deleted." and group `syscheck_entry_deleted`, but not the numeric rule ID. Wazuh's default ID for this alert is 553; that number is **not confirmed by a screenshot** here.

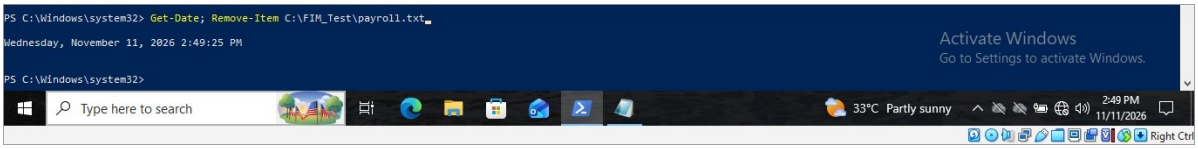


Figure 18 — Deletion command and time (2:49:25 PM).

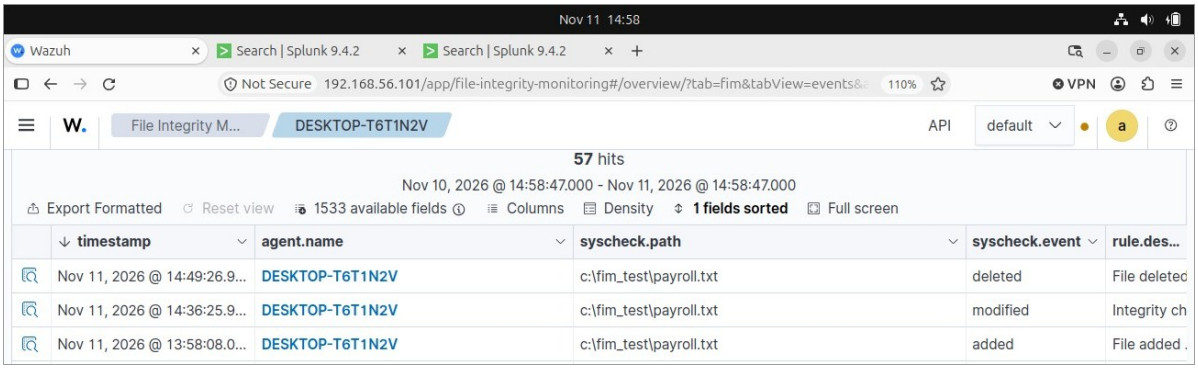


Figure 19 — Three file events for payroll.txt now in the list: added, modified, deleted.

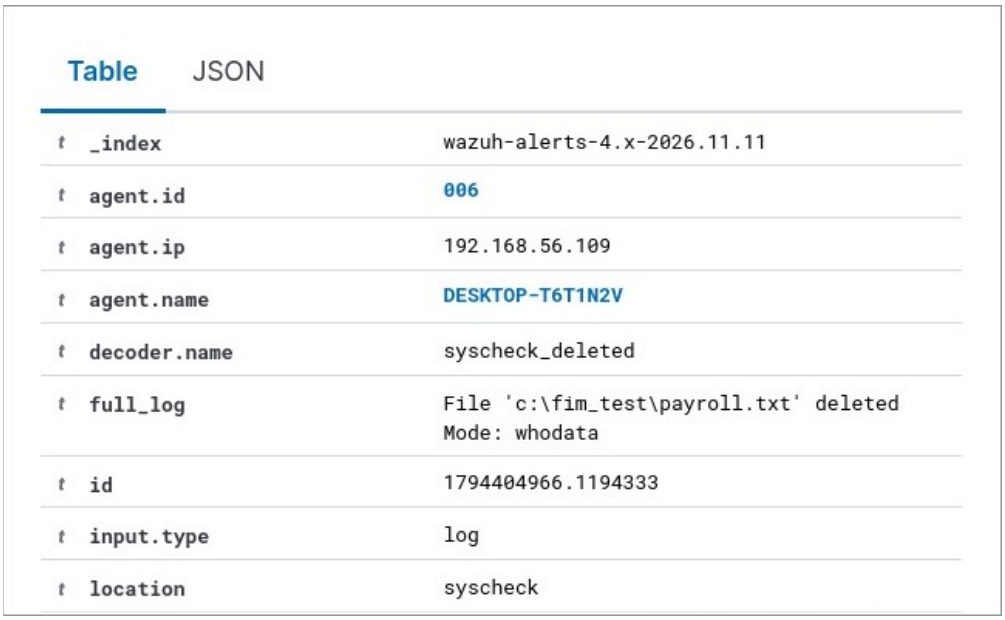
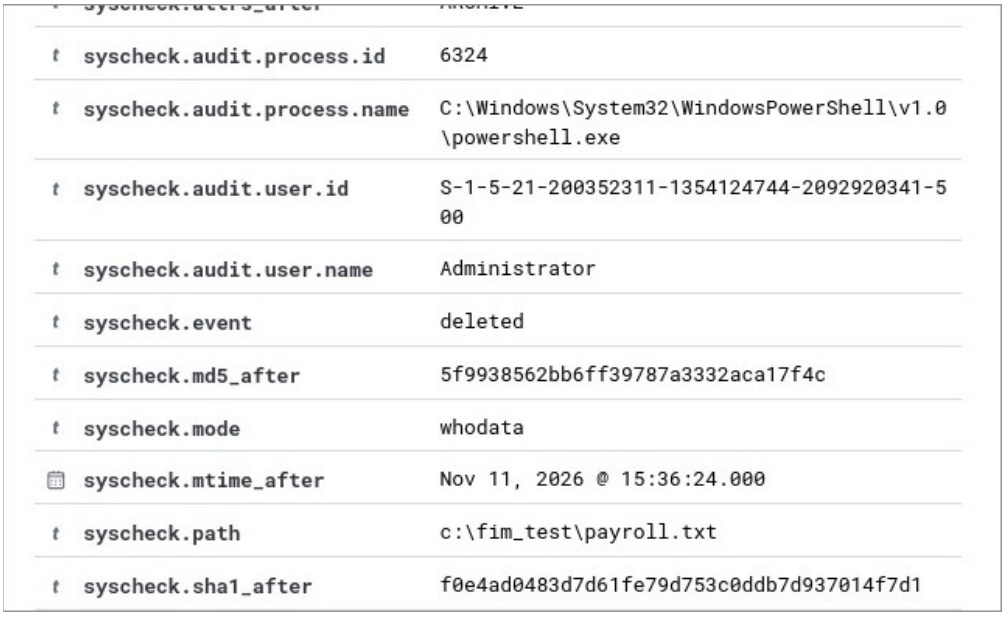


Table JSON

t _index	wazuh-alerts-4.x-2026.11.11
t agent.id	006
t agent.ip	192.168.56.109
t agent.name	DESKTOP-T6T1N2V
t decoder.name	syscheck_deleted
t full_log	File 'c:\fim_test\payroll.txt' deleted Mode: whodata
t id	1794404966.1194333
t input.type	log
t location	syscheck

Figure 20 — Deletion alert: decoder syscheck_deleted, mode whodata.



t syscheck.audit.process.id	6324
t syscheck.audit.process.name	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
t syscheck.audit.user.id	S-1-5-21-200352311-1354124744-2092920341-500
t syscheck.audit.user.name	Administrator
t syscheck.event	deleted
t syscheck.md5_after	5f9938562bb6ff39787a3332aca17f4c
t syscheck.mode	whodata
syscheck.mtime_after	Nov 11, 2026 @ 15:36:24.000
t syscheck.path	c:\fim_test\payroll.txt
t syscheck.sha1_after	f0e4ad0483d7d61fe79d753c0ddb7d937014f7d1

Figure 21 — Deletion alert: user Administrator, powershell.exe PID 6324, last-known MD5 5f9938...17f4c and SHA-1 f0e4ad...f7d1.

The last-known hashes on the deletion alert equal the "after" hashes of the modification, confirming the file was deleted in the same state in which it was last modified.

6.8 Step 8 — Test 4: the unexpected modification

This step simulates a **different program changing a configuration file in a weaker direction**. Two actions were used: create `config.ini` with a secure setting from PowerShell, then change it from a separate Command Prompt.

8a. Create the baseline file

```
Get-Date; "server=10.0.0.5`nmode=secure" | Out-File C:\FIM_Test\config.ini
```

Meaning	The backtick followed by the letter n is PowerShell's newline escape, so the file has two lines: `server=10.0.0.5` and `mode=secure`.

Actual	Created at **3:05:37 PM** (Figure 22). Creation alert at **15:05:38**.

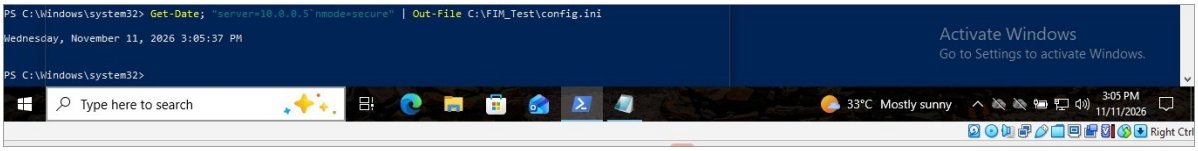


Figure 22 — Creating config.ini from PowerShell (3:05:37 PM).

8b. Change it from cmd.exe

Where	A separate **Administrator Command Prompt** opened from the Start menu.
Command	`echo mode=debug_open>> C:\FIM_Test\config.ini`
Meaning	`echo` prints text; `>>` **appends** it to the file (a single `>` would overwrite).
Actual	Command accepted at about 3:07 PM (taskbar clock in the original screenshot, Figure 23).

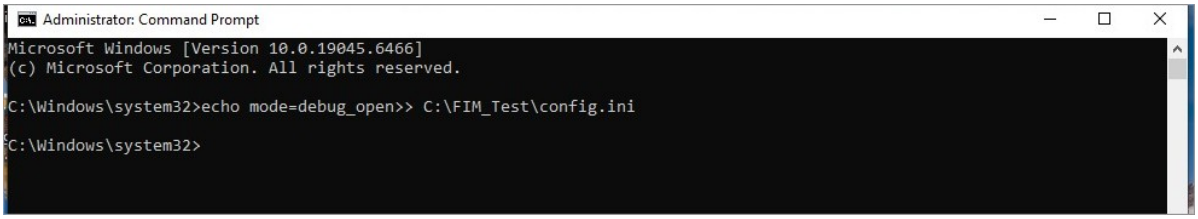


Figure 23 — The appended setting typed into an Administrator Command Prompt (cropped to the window; the original shows 3:07 PM in the taskbar).

8c. Alerts for config.ini

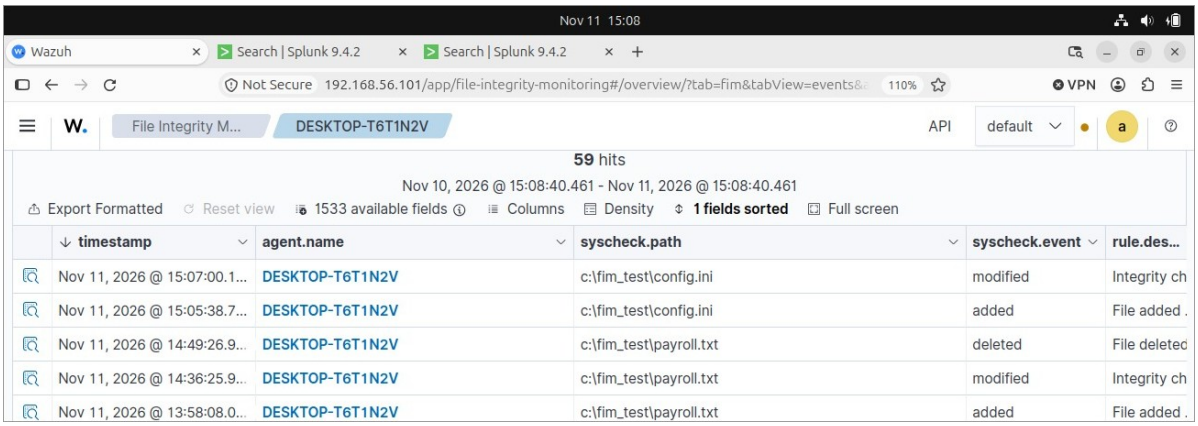


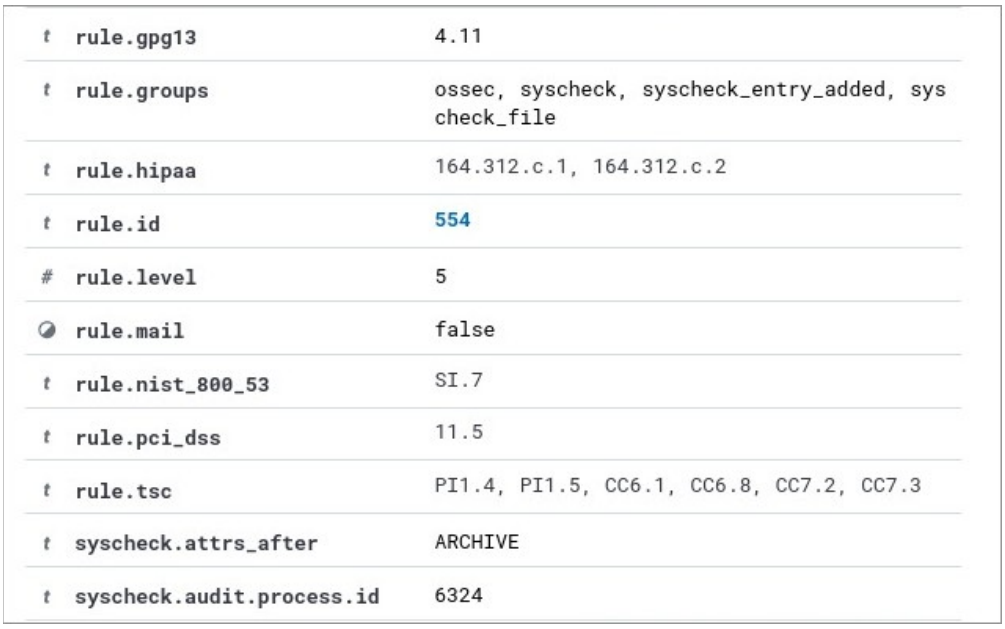
Figure 24 — Events list showing config.ini — added (15:05:38) and modified (15:07:00).

t	rule.mitre.technique	Stored Data Manipulation
t	rule.nist_800_53	SI.7
t	rule.pci_dss	11.5
t	rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
t	syscheck.attrs_after	ARCHIVE
t	syscheck.audit.process.id	6648
t	syscheck.audit.process.name	C:\Windows\System32\cmd.exe
t	syscheck.audit.user.id	S-1-5-21-200352311-1354124744-2092920341-500
t	syscheck.audit.user.name	Administrator
t	syscheck.changed_attributes	size, mtime, md5, sha1, sha256
t	syscheck.diff	--- > mode=debug_open

Figure 25 — Modification alert: process C:\Windows\System32\cmd.exe PID 6648, user Administrator, diff > mode=debug_open.

t	syscheck.audit.user.name	Administrator
t	syscheck.changed_attributes	size, mtime, md5, sha1, sha256
t	syscheck.diff	--- > mode=debug_open
t	syscheck.event	modified
t	syscheck.md5_after	b3171d56f19e6545ca1d6fbf419e6351
t	syscheck.md5_before	74be3bfce53ac98c640253cfb783962c
t	syscheck.mode	whodata
🗓	syscheck.mtime_after	Nov 11, 2026 @ 16:06:59.000
🗓	syscheck.mtime_before	Nov 11, 2026 @ 16:05:37.000
t	syscheck.path	c:\fim_test\config.ini
t	syscheck.sha1_after	d6087f8bbb85e534a0aad54ece3fe7f844d29d46

Figure 26 — Modification alert: before/after hashes for config.ini.



t	rule.gpg13	4.11
t	rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
t	rule.hipaa	164.312.c.1, 164.312.c.2
t	rule.id	554
#	rule.level	5
◕	rule.mail	false
t	rule.nist_800_53	SI.7
t	rule.pci_dss	11.5
t	rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
t	syscheck.attrs_after	ARCHIVE
t	syscheck.audit.process.id	6324

Figure 27 — Creation alert for config.ini — rule 554, audit process ID 6324 (the PowerShell session).



t	syscheck.mode	whodata
📅	syscheck.mtime_after	Nov 11, 2026 @ 16:05:37.000
t	syscheck.path	c:\fim_test\config.ini
t	syscheck.sha1_after	36e6fdd23228b87bd8df67c4ca3365d29192b3e0
t	syscheck.sha256_after	0f1444136a3baa347a2542903de1928835e7467a1fb0ccc4f192b17540f7fd22
#	syscheck.size_after	60
t	syscheck.uid_after	S-1-5-32-544
t	syscheck.uname_after	Administrators
t	syscheck.win_perm_after.allowed	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA,

Figure 28 — Creation alert: SHA-1, SHA-256 and size (60 bytes) of the baseline file.

Table 8 — Evidence for config.ini

Item	Creation	Modification
Alert time	15:05:38	15:07:00
Rule	554 (level 5)	550 (level 7), MITRE T1565.001
Process	PID 6324 (the PowerShell session; process name not captured on this screenshot)	`cmd.exe`, PID **6648**
User	`Administrator`	`Administrator`
Size	60 bytes	60 → 77 bytes
MD5	not captured	`74be3bfce53ac98c640253cfb783962c` → `b3171d56f19e6545ca1d6fbf419e6351`

Item	Creation	Modification
SHA-1	`36e6fdd23228b87bd8df67c4ca3365d29192b3e0`	→ `d6087f8bbb85e534a0aad54ece3fe7f844d29d46`
SHA-256	`0f1444136a3baa347a2542903de1928835e7467a1fb0ccc4f192b17540f7fd22`	→ `611f9a5b8dbd235cb1d573da414e3d98ca66cb67e90fca9036cb00554dd76161`
Diff	—	`> mode=debug_open`
Alert ID	`1794405938.1224164`	`1794406020.1237879`

The creation alert's SHA-256 equals the "before" SHA-256 of the modification alert, so the baseline-to-change chain is continuous. The size change of 17 bytes is consistent with a 15-character line plus a two-byte line ending appended by `cmd.exe`.

6.9 Step 9 — Correlate with Sysmon in Splunk

Where	Splunk (Ubuntu browser), Search & Reporting, time range **Last 24 hours**.
Search	`index=sysmon EventCode=1 Image="*\\cmd.exe" | table _time host EventCode User ParentImage Image CommandLine`
Meaning	Find Sysmon process-creation events whose program is `cmd.exe` and show time, host, user, parent, program and command line.
Result	Three events (Figure 29), interpreted below.

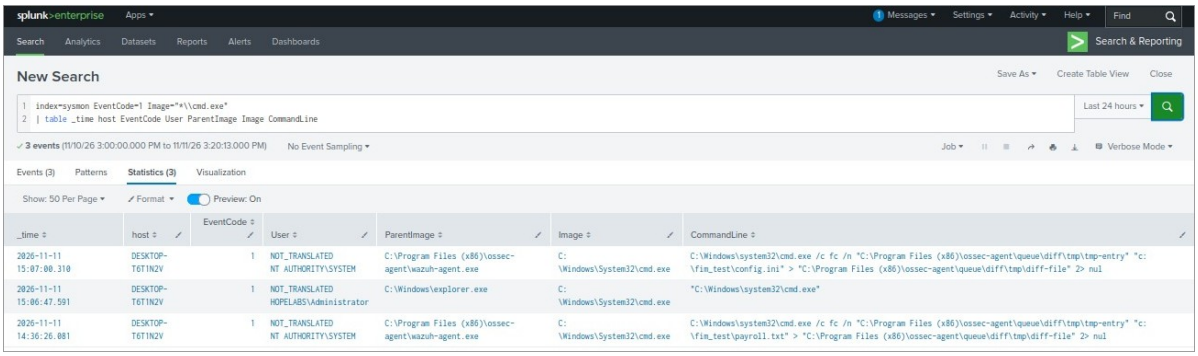


Figure 29 — Splunk: three cmd.exe process-creation events on DESKTOP-T6T1N2V.

Table 9 — Interpretation of the three cmd.exe events

Time	User	Parent process	Interpretation
14:36:26	NT AUTHORITY\SYSTEM	`wazuh-agent.exe`	Wazuh's own helper producing the `payroll.txt` diff (`fc /n ... > diff-file`). **Benign.**
15:06:47	`HOPELABS\Administrator`	`explorer.exe`	The interactive Command Prompt opened by the person at the desktop. **The program used for the change.**
15:07:00	NT AUTHORITY\SYSTEM	`wazuh-agent.exe`	Wazuh's helper producing the `config.ini` diff. **Benign.**

Limitation

Sysmon recorded that `cmd.exe` was **launched**, but not the text typed inside it. The change itself is visible only in the FIM diff.

The Sysmon `ProcessId` was not added to the Splunk table, so the match between PID 6648 (Wazuh) and the Sysmon process was **not verified**; the link rests on user, time (15:06:47 launch, 15:07:00 alert) and program name.

7. Configuration and Detection Logic

7.1 The configuration change (the only one made)

```
<directories check_all="yes" realtime="yes" whodata="yes" report_changes="yes">C:\FIM_Test</directories>
```

Table 10 — Line-by-line explanation

Part	Meaning
`<directories ...>`	Tells syscheck to monitor the listed path.
`check_all="yes"`	Record all attributes: size, permissions, owner, group, modification time, inode, MD5, SHA-1 and SHA-256 (confirmed by the 6003 log message).
`realtime="yes"`	Report changes as they happen instead of waiting for the next scheduled scan.
`whodata="yes"`	Use Windows auditing to attach user and process. Implies real-time.
`report_changes="yes"`	Keep a copy of text files so a diff can be produced. Sensitive content may be copied to the agent's `queue\diff` folder.
`C:\FIM_Test`	The monitored directory.

7.2 Original, changed and recommended

Table 11 — Configuration status

Setting	State	Evidence
`<disabled>no</disabled>` in `<syscheck>`	Original (already enabled)	Notepad screenshots
`<frequency>43200</frequency>`	Original (12 hours)	Notepad screenshots
Default Windows system paths with `restrict=` filters	Original	Notepad screenshots
`windows_audit_interval` 60, `process_priority` 10, `max_eps` 50, synchronization every 5 minutes	Original	Notepad screenshot (end of `<syscheck>`)
`C:\FIM_Test` directory line	**Changed by the author**	Figures 5 and 6
Stray "s" after `</skip_nfs>`	**Corrected by the author**	Figures 5 and 6
Central policy via the manager (`agent.conf`), custom rules, ignore lists	Recommended only	Chapter 12

7.3 Fields used for detection and investigation

Table 12 — Wazuh FIM fields used in this lab

Field	Use
`rule.id`, `rule.level`, `rule.description`, `rule.groups`	Which rule fired, how severe, in which family (`syscheck_entry_added/modified/deleted`).
`syscheck.path`, `syscheck.event`	What file and what happened (added, modified, deleted).
`syscheck.mode`	`whodata` (real-time with attribution) or `scheduled` (periodic scan).

Field	Use
`syscheck.audit.user.name`, `.user.id`	Who made the change (SID ending `-500` is the well-known built-in Administrator RID).
`syscheck.audit.process.name`, `.process.id`	Which program, which instance.
`syscheck.md5_before/after`, `sha1_*`, `sha256_*`	Content fingerprints before and after.
`syscheck.size_before/after`, `syscheck.changed_attributes`	Which attributes changed.
`syscheck.diff`	The text added or removed (needs `report_changes`).
`rule.mitre.id`, `.tactic`, `.technique`	ATT&CK mapping (T1565.001 on file modification).

7.4 Queries used

Table 13 — Searches used or shown in the evidence

Tool	Query	Purpose
Wazuh dashboard (DQL)	`syscheck.path: *config.ini* AND syscheck.event: added`	Find the creation alert for `config.ini` (shown in the filter bar). Active filter pills: `manager.name: Ubuntu`, `rule.groups: syscheck`, `agent.id: 006`.
Splunk (SPL)	`index=sysmon EventCode=1 Image="*\\cmd.exe" | table _time host EventCode User ParentImage Image CommandLine`	Correlate `cmd.exe` launches.
PowerShell	`Select-String` against `ossec.log`	Confirm FIM started.

7.5 Example improvement (NOT tested in this lab)

A custom rule could raise the severity when a configuration file in the monitored folder is changed by `cmd.exe`. This is an **untested illustration**: it must be placed in the manager's local rules, validated with the manager's rule-testing tool, and the rule ID chosen from the custom range.

```
<group name="syscheck,local_fim,">
  <rule id="100900" level="10">
    <if_sid>550</if_sid>
    <field name="syscheck.path" type="pcre2">(?i)fim_test\\config\.ini</field>
    <field name="syscheck.audit.process.name" type="pcre2">(?i)cmd\.exe</field>
    <description>Lab example: config.ini modified by cmd.exe</description>
  </rule>
</group>
```

8. Testing and Results

8.1 Test scenarios

Table 14 — Test scenarios, expectations and outcomes

ID	Scenario	Expected	Actual	Status
T1	FIM enabled for `C:\FIM_Test`	Log shows monitoring, real-time and who-data started	Messages 6003, 6012, 6019 (Fig. 8)	Pass
T2	Create `payroll.txt`	Rule 554 with user, process, hashes	Rule 554; `Administrator`; `powershell.exe` 6324; hashes recorded	Pass
T3	Modify `payroll.txt`	Rule 550 with before/after hashes and diff	Rule 550, level 7; hashes and diff recorded	Pass
T4	Delete `payroll.txt`	Deletion alert with last-known hashes	Description "File deleted."; hashes match T3 after-state; numeric rule ID not captured	Pass (partial evidence)
T5	Create `config.ini`	Rule 554	Rule 554 at 15:05:38; PID 6324	Pass (process name not captured)
T6	Modify `config.ini` from `cmd.exe`	Rule 550 naming the different process	Rule 550; `cmd.exe` PID 6648; diff `mode=debug_open`	Pass
T7	Correlate in Splunk	Process-creation event for the `cmd.exe`	Event at 15:06:47, parent `explorer.exe`	Pass (PID match not verified)

8.2 File timeline (required deliverable)

Times are the Wazuh alert timestamps, which matched the Windows VM clock to within about two seconds.

Table 15 — File timeline

Time	File	Event	Rule	User	Process (PID)	Hash after
13:58:08	`payroll.txt`	added	554	Administrator	`powershell.exe` (6324)	SHA-256 `eda2b63c…06a9a`
14:36:25	`payroll.txt`	modified	550	Administrator	`powershell.exe` (6324)	SHA-256 `dcb2bfa6…b9f1`
14:49:26	`payroll.txt`	deleted	not shown	Administrator	`powershell.exe` (6324)	last known MD5 `5f993856…7f4c`
15:05:38	`config.ini`	added	554	Administrator	PID 6324 (PowerShell)	SHA-256 `0f144413…7fd22`
15:06:47	—	(Sysmon) `cmd.exe` launched	—	`HOPELABS\Administrator`	`cmd.exe`, parent `explorer.exe`	—

Time	File	Event	Rule	User	Process (PID)	Hash after
15:07:00	`config.ini`	modified	550	Administrator	`cmd.exe` (6648)	SHA-256 `611f9a5b…6161`

8.3 What succeeded, failed and remains untested

- **Succeeded:** detection of all four file events in real time with who-data; hash and diff capture; correlation with Sysmon.
- **Not captured:** the numeric rule ID of the deletion alert; the process name on the `config.ini` creation alert; the MD5 on that alert; the Sysmon `ProcessId`.
- **Untested:** behaviour when a file is renamed, when permissions or ownership change, when a different (non-administrator) user makes the change, when the agent is offline, and large-folder performance.

9. Evidence and Screenshot Analysis

The screenshots are placed in Chapter 6 beside the steps they support. This chapter evaluates what the evidence proves and where it is weak.

9.1 Evidence handling

- The original screenshots are retained unchanged as the source evidence (see the register in Appendix C).
- The figures in this report are presentation copies made by **cropping** only: browser docks, status bars, header strips and empty space were removed; a thin grey border was added. No pixel content was edited, redrawn, recoloured or annotated.
- No redaction was applied. The images contain host names, private IP addresses and Windows SIDs of a disposable, isolated lab; none are real-world identifiers.
- Two figures are diagrams drawn for this report (Figures 2 and 3); they are not evidence.

9.2 Background alerts seen alongside the test

Early views of the dashboard (Figure 30) were dominated by registry alerts with `syscheck.mode: scheduled`, dated about 04:06–04:07, hours before the test. Examples were rule 752 "Registry Value Entry Added to the System", rule 594 "Registry Key Integrity Checksum Changed" and rule 751 for deleted registry values, covering Windows Defender policy and firewall-rule keys changed by `SYSTEM` or `LOCAL SERVICE`. The Events count was 56 before the second `payroll.txt` event, so most results were unrelated to the test.

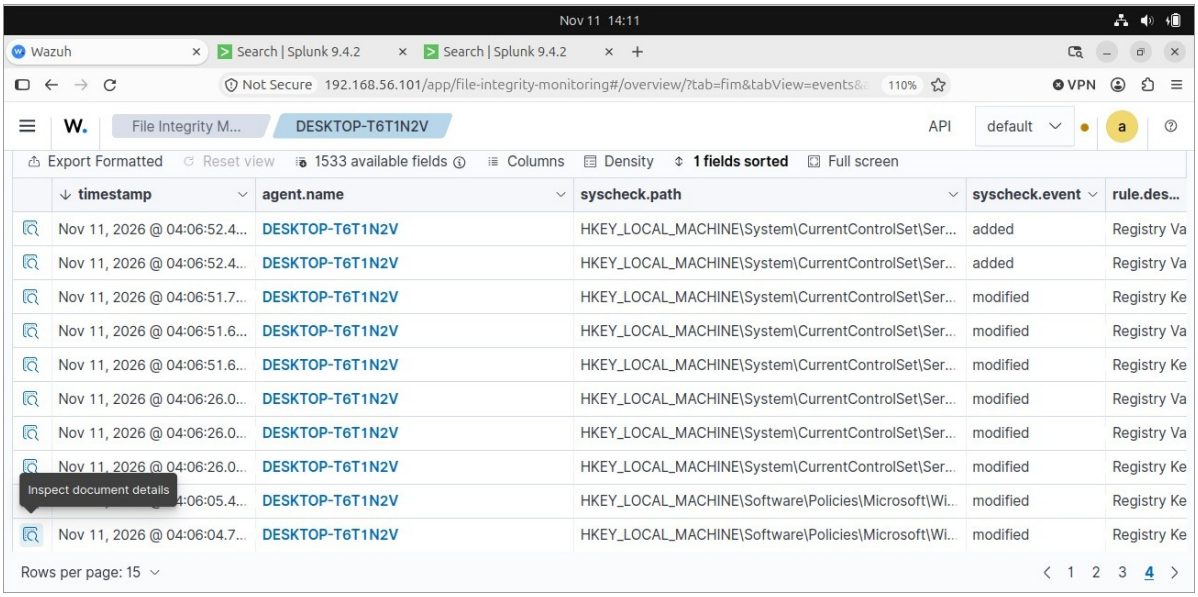


Figure 30 — Scheduled-mode registry alerts (04:06–04:07) listed beside the test events; they are background noise, not part of the test.

9.3 What each evidence group proves

Table 16 — Evidence assessment

Figures	Shows	Proves	Limits
4	Folder creation	Test folder existed at 1:29 PM	Does not show the folder was empty

Figures	Shows	Proves	Limits
5–8	Config edit, service state, log	The monitored path and who-data engine were active before testing	Wazuh version not shown
9–12	Create `payroll.txt`	Add alert with user, process, hashes	Hash values read from the screen; compare with originals
13–17	Modify `payroll.txt`	Content change with before/after hashes and diff	The content diff is truncated on screen
18–21	Delete `payroll.txt`	Deletion attributed to the same process	Numeric rule ID not visible
22–28	`config.ini` create and modify	Different process (`cmd.exe` 6648) changed the file	Creation-alert process name not captured
29	Splunk `cmd.exe` events	Interactive launch from `explorer.exe`; helper noise identified	PID not shown; typed text not logged
30	Background alerts	Noise exists in the same view	Cause of the 04:06 batch not investigated

10. Troubleshooting and Root-Cause Analysis

Table 17 — Problems encountered

#	Symptom	Likely cause	Action	Status
P1	Stray "s" after `</skip_nfs>` in `ossec.conf`	Typing/paste slip while editing	Deleted the character, saved, restarted agent; service `Running`	Resolved. Effect of the original text not tested
P2	The first dashboard views showed registry alerts, not `payroll.txt`	Many scheduled-mode registry alerts in the 24-hour window; file alerts were on the same list but not visible	Searched by path (`*fim_test*`, `*config.ini*`) and widened the time range to 24 hours	Resolved
P3	`mtime_after` shows one hour later than the alert time (for example 14:58:07 vs 13:58:08)	Not investigated. Possibly time-zone handling of Windows file times in the dashboard	Used the alert timestamp, which matched the VM clock	**Unresolved**
P4	Registry alerts dated about 04:06 appeared in the dashboard	Scheduled-mode events; the cause (for example an earlier scan) was not investigated. They predate the 13:52 agent restart	Treated as background	**Cause unknown**
P5	Two `cmd.exe` events in Splunk looked like scripted activity	They were launched by `wazuh-agent.exe` as SYSTEM to build diffs (`fc /n ...`)	Compared parent image and command line; excluded as benign	Resolved
P6	Typed text was not visible in Sysmon	Sysmon Event ID 1 logs launch, not input	Used the FIM diff for content	Explained

> **Note**
> A useful troubleshooting order for "I made a change but see no alert": (1) confirm the change happened; (2) check the agent log for the monitored path; (3) confirm the agent is connected; (4) search the dashboard by path and widen the time range; (5) only then suspect rules.

11. Security Findings and Risk Analysis

11.1 Findings

Table 18 — Findings and confidence

ID	Finding	Basis	Confidence	Likely impact if real
F1	A configuration file was changed to a weaker setting (`mode=secure` followed by `mode=debug_open`) by a different program than the one that created it.	Rule 550 alert, diff, `cmd.exe` PID 6648; Sysmon launch	Confirmed (simulation)	Reduced security posture or hidden debug access; severity depends on the real file
F2	All actions used the built-in Administrator account (SID ending `-500`).	Who-data user fields	Confirmed	No separation of duties; harder to distinguish normal from malicious use
F3	Wazuh's own helper processes create `cmd.exe` events that resemble scripted activity.	Splunk rows at 14:36:26 and 15:07:00	Confirmed	False-positive risk in process-based hunting
F4	FIM shows file state changes but not intent or typed input.	Absence of typed text in Sysmon; need for context	Confirmed	Analyst must investigate every alert
F5	`report_changes` stores file content copies on the endpoint.	Configuration; diff displayed	Config-based; not inspected	Possible exposure of sensitive text
F6	Scheduled FIM produced many background registry alerts.	Figure 30	Confirmed	Alert fatigue

11.2 Analyst notes — the six analyst questions

Answers refer to the unexpected-modification scenario (`config.ini`); the `payroll.txt` lifecycle is summarised in Section 8.2.

What happened?	`config.ini` was created at 15:05:38 (60 bytes) and modified at 15:07:00 (77 bytes). A line `mode=debug_open` was appended beneath `mode=secure`, weakening the setting.
How was it detected?	Wazuh syscheck in real-time mode with who-data produced rule 554 (level 5) then rule 550 (level 7, MITRE T1565.001) alerts within about one to two seconds.
What evidence supports the finding?	Alert ID `1794406020.1237879`; `syscheck.diff > mode=debug_open`; changed attributes `size, mtime, md5, sha1, sha256`; SHA-256 `0f144413…7fd22` → `611f9a5b…6161`; Sysmon Event ID 1 for the `cmd.exe` launch at 15:06:47 with parent `explorer.exe`.
Which host and account?	Host `DESKTOP-T6T1N2V` (192.168.56.109, agent 006). Wazuh: `Administrator` (SID ending `-500`). Sysmon: `HOPELABS\Administrator`. The two sources display the account name differently; the SID suggests the built-in Administrator, but the Sysmon SID was not captured.

What else should be checked?	• Windows Security log: logon events for that session, and object-access (4663) events that who-data uses. • PowerShell logging (4104) for the creator process. • Sysmon file-create events (Event ID 11), only if the Sysmon configuration collects them. • The process tree and command line of `cmd.exe` PID 6648 and its siblings. • Other hosts and Splunk for the same hash or file name. • Who was signed in at that time, and any change ticket.
What action in a real SOC?	1. Check whether a change ticket or an authorised admin explains it. 2. Contact the account owner; confirm the activity. 3. If unexplained: preserve evidence (alert, hashes, process data), restore the file from the known-good version, reset the account's credentials, and isolate the host if other suspicious activity appears. 4. Hunt for the same behaviour elsewhere. 5. Tune rules and record the lesson learned.

11.3 Why context matters

Taken alone, a rule 550 alert says only that bytes changed. Context turns it into a decision: **which file** (a test file or a service configuration?), **who and which program** (an administrator using the usual tool or an unusual process?), **when** (inside a change window?), **what changed** (a harmless comment or a weakened setting?), and **what else happened** around it. In this lab the diff showed a security-relevant change, while the same `cmd.exe` image appearing under `wazuh-agent.exe` was harmless. Only context separated the two.

12. Mitigation and Remediation

12.1 Implemented in the lab

Table 19 — Controls actually implemented

Control	Evidence
Monitoring confined to a dedicated test folder	Figures 4 and 8
Backup of `ossec.conf` before editing	Figure 7 (`Copy-Item` command)
Real-time monitoring with who-data and content diff	Figures 5–8
Configuration fault (stray character) corrected	Figures 5 and 6
Detection verified end-to-end for create, modify, delete	Figures 9–21

> **Note**
> No remediation of the simulated change (restoring `config.ini` to `mode=secure`) was performed or documented in the lab. The recommendations below are **not implemented**.

12.2 Recommendations (not implemented)

R1 — Restrict who can change important files

Objective	Reduce the chance that any process can weaken a configuration file.
Implementation	Set NTFS permissions so only a named service account or change group has write access to monitored configuration paths; use least-privilege accounts for day-to-day work instead of the built-in Administrator.
Benefit	Prevents or limits the change and makes who-data attribution more meaningful.
Operational impact	Administrators must request or elevate access for legitimate changes.
Verification	Repeat the `cmd.exe` append test as a standard user; expect "Access is denied" and no rule 550 alert (or a failure event in the Security log).
Residual risk	Privileged attackers or compromised administrator accounts can still change files; FIM remains necessary.

R2 — Raise severity for risky combinations

Objective	Make the dangerous change stand out from routine rule 550 alerts.
Implementation	Add a custom rule such as the untested example in Section 7.5, validate it with the manager's rule-testing tool, then deploy.
Benefit	Faster triage of config changes made by unusual processes.
Operational impact	Possible new false positives for legitimate administrator use of `cmd.exe`.
Verification	Re-run the Step 8 test; expect the new rule ID and level.
Residual risk	Attackers can use other programs; process names are easy to rename.

R3 — Reduce background FIM noise

Objective	Limit alert fatigue from scheduled registry and system-path events.
Implementation	Review the default monitored registry keys, lengthen or shorten scan frequency deliberately, and add `ignore` entries only for items proven benign.
Benefit	Clearer dashboards; real changes stand out.
Operational impact	Ignored paths are no longer watched; each exclusion needs justification.
Verification	Compare the number of `syscheck` alerts per day before and after the tuning.
Residual risk	Over-tuning can hide real attacks.

R4 — Automate the FIM-to-process correlation

Objective	Avoid the manual pivot from Wazuh to Splunk.
Implementation	Create a Splunk saved search or dashboard that lists `cmd.exe`/`powershell.exe` launches near the time of FIM modifications, and add `ProcessId` and `CommandLine` columns. Exclude `wazuh-agent.exe` parents from "suspicious" lists after review.
Benefit	Faster, repeatable investigations; PID-level matching.
Operational impact	Query maintenance.
Verification	Repeat Step 8; the saved search should show the 15:06:47-style launch with PID.
Residual risk	Sysmon still does not log typed input.

R5 — Protect stored diffs

Objective	Prevent file contents copied by `report_changes` from leaking.
Implementation	Enable `report_changes` only for non-sensitive text files; restrict access to the agent's `queue\diff` folder; avoid monitoring files that contain secrets with this option.
Benefit	Lower data-exposure risk.
Operational impact	No diff for excluded files.
Verification	Inspect the agent folder permissions and confirm no diffs exist for excluded paths.
Residual risk	Hashes still reveal that a change occurred.

R6 — Make time handling consistent

Objective	Avoid confusion such as the one-hour `mtime` offset and 04:06 batch.
Implementation	Synchronise clocks (NTP) on all VMs and set the dashboard time zone consistently; investigate why `mtime_after` displays one hour later than the alert time.
Benefit	Reliable timelines.
Operational impact	Minimal.
Verification	Create a file and compare Windows time, file modification time and alert time.
Residual risk	Misconfigured clocks can still occur after reboots.

13. What I Learned

This reflection is written in the first person and is limited to what the Day 9 work actually involved. The lab was completed with step-by-step guidance, so I describe what I now understand and what I still need to practise on my own.

Concepts

I learned that a FIM alert is a statement about a **change in state**, not a statement about an attack. A hash proves content changed; it cannot say why. I also learned the difference between scheduled scanning and real-time monitoring, and that who-data is what makes an alert useful, because it adds the user and the process.

Practical skills

I edited the Wazuh agent configuration, restarted the service, and confirmed through the agent log that the path was actually monitored. I generated create, modify and delete events with PowerShell and `cmd.exe`, and found the resulting alerts in the Wazuh dashboard by filtering on the file path.

Troubleshooting

A stray character in an XML configuration is easy to miss and potentially disruptive. I learned to back up before editing and to verify the result with the service status and the log, rather than assuming the change worked. I also learned to widen the time range and filter by path when a dashboard is crowded with unrelated alerts.

Evidence analysis

Reading the before/after hashes, sizes and diffs showed me how one alert connects to the next: the "before" hash of one alert matched the "after" hash of the previous one. I also saw that screenshots can leave gaps (the deletion rule ID, the process name on one creation alert), so I note gaps instead of filling them in.

Detection engineering and security reasoning

Comparing the PowerShell creation with the `cmd.exe` modification taught me to look at **which process** changed a file, not only that it changed. The Splunk results showed that two of three `cmd.exe` events were Wazuh's own helper, which taught me to check parent process and command line before judging a process as suspicious.

Areas for further development

- Writing, testing and deploying custom Wazuh rules (Section 7.5 is untested).
- Testing FIM on renames, permission changes and changes by non-administrator users.
- Understanding how Sysmon data reaches Splunk in this lab, and adding `ProcessId` to my correlation searches.
- Investigating the unexplained one-hour `mtime` offset and the early-morning registry alert batch.

14. Practical Knowledge and Competencies

"Performed (guided)" means I carried out the steps myself following written step-by-step instructions. "Studied" means the concept was explained and observed but not independently designed.

Table 20 — Skills mapped to lab tasks

Skill	Lab task	Tool	Evidence	Level
Endpoint monitoring configuration	Add `C:\FIM_Test` with real-time and who-data	Wazuh agent, Notepad	Figures 5–8	Performed (guided)
Log-based verification	Confirm FIM started	PowerShell `Select-String`	Figure 8	Performed (guided)
Event generation	Create, modify, delete files	PowerShell, `cmd.exe`	Figures 9, 13, 18, 22, 23	Performed (guided)
Alert triage	Locate alerts and read fields	Wazuh dashboard	Figures 10–21, 24–28	Performed (guided)
Hash and diff analysis	Compare before/after values	Wazuh alert fields	Tables in Chapter 6	Performed (guided)
Cross-source correlation	Match FIM alert to process launch	Splunk, Sysmon	Figure 29	Performed (guided)
False-positive recognition	Identify `wazuh-agent.exe` helper events	Splunk	Figure 29, Table 9	Studied / observed
Detection engineering	Custom rule idea	Wazuh rules	Section 7.5	Studied (untested)
Report writing	Timeline, notes, risk analysis	This document	All chapters	Performed

14.1 Suggested next exercises

- Rename and move files inside `C:\FIM_Test`; record the alerts.
- Change permissions or ownership with `icacls`; compare the alerts.
- Make a change as a non-administrator user and compare who-data.
- Write and test a custom rule using the manager's rule-testing tool.
- Build a Splunk dashboard that joins FIM events and Sysmon launches.

15. Reproduction Guide

This standalone guide lets another learner repeat the experiment. "Windows VM" means the monitored endpoint; "Ubuntu VM" means the Wazuh/Splunk server. Run all PowerShell commands as **Administrator**.

15.1 Initial state

- Wazuh manager and dashboard running; the Windows VM enrolled and showing **Active**.
- Wazuh agent installed on the Windows VM at `C:\Program Files (x86)\ossec-agent\`.
- Optional: Sysmon on the Windows VM with its events searchable in Splunk.
- A VirtualBox snapshot of both VMs (recommended).

15.2 Setup

1. **Windows VM:** create the folder. `New-Item -ItemType Directory C:\FIM_Test` then `Test-Path C:\FIM_Test`. Expect `True`.
2. **Windows VM:** back up the config: `Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" "C:\Program Files (x86)\ossec-agent\ossec.conf.bak"`.
3. **Windows VM:** open it: `notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"`.
4. Find `<syscheck>`; make sure `<disabled>no</disabled>`. Under `<frequency>` add the line below. Check the file contains no stray characters. Save.
5. **Windows VM:** `Restart-Service -Name wazuh` then `Get-Service -Name wazuh`. Expect `Running`.
6. **Windows VM:** `Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.log" -Pattern "syscheck|FIM_Test|whodata|real-time" | Select-Object -Last 10`. Expect the three messages described in Section 6.4. **Do not continue until they appear.**

```
<directories check_all="yes" realtime="yes" whodata="yes" report_changes="yes">C:\FIM_Test</directories>
```

15.3 Test activities

Table 21 — Commands to run and what to expect

Step	Command (Windows VM)	Expected alert
Create	`Get-Date; "Employee salary list v1" | Out-File C:\FIM_Test\payroll.txt`	Rule 554, added
Modify	`Get-Date; Add-Content C:\FIM_Test\payroll.txt "Salary changed by me"`	Rule 550, hashes changed, diff
Delete	`Get-Date; Remove-Item C:\FIM_Test\payroll.txt`	Deleted alert (default rule 553)
Baseline	Get-Date; "server=10.0.0.5`nmode=secure" | Out-File C:\FIM_Test\config.ini	Rule 554, added
Unexpected change	In a **new Administrator Command Prompt**: `echo mode=debug_open>> C:\FIM_Test\config.ini`	Rule 550, process `cmd.exe`

> **Note**
> In the "Baseline" row the character before `nmode` is a **backtick**, not an apostrophe. Note the time printed by `Get-Date` for each step.

15.4 Verification and evidence collection

1. **Ubuntu VM:** open the Wazuh dashboard, then Integrity monitoring for the agent, then the **Events** tab.
2. Set the time range to **Last 24 hours**. Search `syscheck.path: *fim_test*` (the path is stored in lower case).
3. Expand each alert and record: time, `rule.id`, `syscheck.path`, `syscheck.event`, `syscheck.audit.user.name`, `syscheck.audit.process.name` and ID, size and the MD5/SHA-1/SHA-256 before and after, `syscheck.diff`.
4. Compare each alert time with the `Get-Date` time (expect a difference of seconds).
5. **Splunk:** run `index=sysmon EventCode=1 Image="*\\cmd.exe" | table _time host User ProcessId ParentImage Image CommandLine`. Identify which events came from `explorer.exe` (interactive) and which from `wazuh-agent.exe` (benign helper). Match the interactive event's `ProcessId` with `syscheck.audit.process.id`.
6. Take a full-screen screenshot of each step and keep the originals unedited.

15.5 Troubleshooting

Table 22 — Common problems

Symptom	What to check
Service will not start after the edit	Restore `ossec.conf.bak`; look for unclosed tags or stray characters; restart.
No 6003/6012/6019 lines in the log	Wrong folder path; the line is not inside `<syscheck>`; the file was not saved; the service was not restarted.
Alert appears with `scheduled` mode, no user/process	The directory line lacks `whodata="yes"`, or auditing could not be set.
Cannot find the alert	Widen the time range; search by path; confirm the agent is Active; check the agent log.
No diff on modification	`report_changes="yes"` missing; file is not text.

15.6 Cleanup and rollback (documented, not executed in this lab)

```
Remove-Item C:\FIM_Test\config.ini            # generates a "deleted" alert
Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf.bak" "C:\Program Files
(x86)\ossec-agent\ossec.conf" -Force
Restart-Service -Name wazuh
Remove-Item C:\FIM_Test                        # optional, after monitoring is
removed
```

16. Limitations and Future Improvements

16.1 Limits of the experiment

- One endpoint, one folder, one account (Administrator); no tests of other users, renames or permission changes.
- The "unexpected change" was staged; real attackers behave differently and may disable or evade monitoring.
- Only text files were changed; binary files, large files and bursts of changes were not tested.
- The Wazuh version, the full manager configuration and the Sysmon configuration were not captured in the evidence.

16.2 Timestamp and display anomalies

- The `mtime_after` field showed times exactly one hour after the matching alert time for every event. The cause was not investigated, so alert timestamps were used for the timeline.
- A batch of scheduled-mode registry alerts is dated about 04:06–04:07, hours before the test; its cause was not investigated.

16.3 Evidence gaps

- Numeric rule ID of the deletion alert (default 553; not captured).
- Process name and MD5 on the `config.ini` creation alert.
- Sysmon `ProcessId`, so PID 6648 was not matched to Sysmon.
- Wazuh version, `ossec.log` after the test, and the full Sysmon configuration.

16.4 False positives and false negatives

False positives seen: scheduled registry alerts and the `wazuh-agent.exe` helper `cmd.exe` events. **False negatives not tested:** changes made while the agent is stopped, changes to unmonitored paths, and an attacker who modifies a file and restores its content and timestamps before the next scan (real-time mode narrows this window).

16.5 Future improvements

See Chapter 12 (R1–R6) and the next exercises in Section 14.1. How the limits affect the conclusions: the findings in Chapter 11 are valid for this controlled scenario and should not be generalised to production environments without further testing.

17. Conclusion

Wazuh FIM on a dedicated test folder detected every create, modify and delete action in real time, and with who-data attributed each to a user and process. Hashes and diffs gave evidence of exactly what changed. A staged change from `cmd.exe` that weakened `config.ini` was detected as a rule 550 alert and traced through Sysmon to an interactive Command Prompt started from `explorer.exe`.

The lab also showed the limits of FIM: it records that a file changed, not why; it can be noisy; and typed input is invisible without other data sources. Context (file importance, user, process, time and related events) is what lets an analyst decide whether a change is acceptable.

The evidence has gaps, listed in Section 16.3, and the scenario was simulated, so the conclusions apply to this lab. The next steps are to test renames, permission changes and non-administrator users, write and test a custom rule, and automate the correlation between FIM and Sysmon.

A. Appendix A — Command History

Commands actually run during the lab, in order. Times are from `Get-Date` output or the taskbar clock.

Table 23 — Commands executed

Time	Command
~13:29	`New-Item -ItemType Directory C:\FIM_Test`; `Test-Path C:\FIM_Test`
~13:33	`Copy-Item "…\ossec.conf" "…\ossec.conf.bak"`; `notepad "…\ossec.conf"`
~13:52	`Restart-Service -Name wazuh`; `Get-Service -Name wazuh`
~13:55	`Select-String -Path "…\ossec.log" -Pattern "syscheck|FIM_Test|whodata|real-time" | Select-Object -Last 10`
13:58:07	`Get-Date; "Employee salary list v1" | Out-File C:\FIM_Test\payroll.txt`
14:36:24	`Get-Date; Add-Content C:\FIM_Test\payroll.txt "Salary changed by me"`
14:49:25	`Get-Date; Remove-Item C:\FIM_Test\payroll.txt`
15:05:37	Get-Date; "server=10.0.0.5`nmode=secure" | Out-File C:\FIM_Test\config.ini
~15:07	(cmd.exe) `echo mode=debug_open>> C:\FIM_Test\config.ini`
~15:20	Splunk: `index=sysmon EventCode=1 Image="*\\cmd.exe" | table _time host EventCode User ParentImage Image CommandLine`

B. Appendix B — Configuration and Log Excerpts

Final FIM directory line:

```
<directories check_all="yes" realtime="yes" whodata="yes" report_changes="yes">C:\
FIM_Test</directories>
```

Agent log messages (transcribed from Figure 8):

```
2026/11/11 13:52:47 wazuh-agent: INFO: (6003): Monitoring path: 'c:\fim_test', with
options 'size | permissions |
  owner | group | mtime | inode | hash_md5 | hash_sha1 | hash_sha256 | attributes |
report_changes | whodata'.
2026/11/11 13:54:02 wazuh-agent: INFO: (6012): Real-time file integrity monitoring
started.
2026/11/11 13:54:08 wazuh-agent: INFO: (6019): File integrity monitoring real-time
Whodata engine started.
```

Alert identifiers:

Table 24 — Alert IDs

Event	Alert ID
payroll.txt added	1794401888.1058407
payroll.txt modified	1794404185.1153677
payroll.txt deleted	1794404966.1194333
config.ini added	1794405938.1224164
config.ini modified	1794406020.1237879

C. Appendix C — Evidence Register

Each figure and the original source file in the author's upload set. The originals are unchanged; the figures are cropped copies (Section 9.1).

Table 25 — Evidence register

Figure	Description	Source file
1	Day 9 lab brief (source: programme PDF, page 11 and following), reproduced as the reference for this report's structure.	1000501727.jpg
2	The six stages between a file change and an analyst reading the alert.	Generated diagram (not evidence)
3	Laboratory architecture and data flow.	Generated diagram (not evidence)
4	Creating and confirming the C:\FIM_Test directory.	1000501739.jpg
5	Edited <syscheck> block before saving; the stray "s" is visible after </skip_nfs> at the top.	1000501756.jpg
6	After correction: </skip_nfs> ends cleanly and the new C:\FIM_Test line sits under <frequency>.	1000501759.jpg
7	Backup, edit, restart and service status.	1000501762.jpg
8	Agent log confirming that c:\fim_test is monitored in real-time with who-data.	1000501765.jpg
9	Creation command and its Get-Date output (cropped from a longer session screenshot).	1000501768.jpg
10	Events list: c:\fim_test\payroll.txt — added — rule 554 — at the top of the list.	1000501919.jpg
11	Creation alert: process powershell.exe (PID 6324), user Administrator, event added, mode whodata.	1000501922.jpg
12	Creation alert: path, SHA-1, SHA-256, size 52 bytes, owner group Administrators.	1000501924.jpg
13	Modification command and time (2:36:24 PM).	1000501951.jpg
14	Events list: a "modified" row (rule 550, level 7) above the earlier "added" row.	1000501992.jpg
15	The alert's full_log: size changed from 52 to 96, with before/after checksums.	1000501993.jpg
16	Who-data and diff: same PID 6324, changed attributes size, mtime, md5, sha1, sha256.	1000501997.jpg
17	Before/after hashes and sizes for the modification.	1000501999.jpg
18	Deletion command and time (2:49:25 PM).	1000502026.jpg
19	Three file events for payroll.txt now in the list: added, modified, deleted.	1000502050.jpg
20	Deletion alert: decoder syscheck_deleted, mode whodata.	1000502051.jpg

Figure	Description	Source file
21	Deletion alert: user Administrator, powershell.exe PID 6324, last-known MD5 5f9938...17f4c and SHA-1 f0e4ad...f7d1.	1000502057.jpg
22	Creating config.ini from PowerShell (3:05:37 PM).	1000502064.jpg
23	The appended setting typed into an Administrator Command Prompt (cropped to the window; the original shows 3:07 PM in the taskbar).	1000502067.jpg
24	Events list showing config.ini — added (15:05:38) and modified (15:07:00).	1000502090.jpg
25	Modification alert: process C:\Windows\System32\cmd.exe PID 6648, user Administrator, diff > mode=debug_open.	1000502095.jpg
26	Modification alert: before/after hashes for config.ini.	1000502149.jpg
27	Creation alert for config.ini — rule 554, audit process ID 6324 (the PowerShell session).	1000502176.jpg
28	Creation alert: SHA-1, SHA-256 and size (60 bytes) of the baseline file.	1000502177.jpg
29	Splunk: three cmd.exe process-creation events on DESKTOP-T6T1N2V.	1000502114.jpg
30	Scheduled-mode registry alerts (04:06–04:07) listed beside the test events; they are background noise, not part of the test.	1000501826.jpg

D. Appendix D — Glossary

Table 26 — Glossary

Term	Meaning
ACL	Access control list: who may read or write a file.
Agent	Software on an endpoint that sends data to the Wazuh manager.
Alert	A stored result produced when an event matches a rule.
Baseline	The remembered "normal" state of a file.
Decoder	Manager component that splits a raw event into named fields.
DQL	Dashboard query language used in the Wazuh dashboard search bar.
FIM	File Integrity Monitoring.
Hash	A fingerprint of file content (MD5, SHA-1, SHA-256).
MITRE ATT&CK	A catalogue of adversary tactics and techniques.
PID	Process ID: a number identifying a running program instance.
SID	Security identifier: a unique ID for a Windows account or group.
SIEM	Security information and event management platform.
SOC	Security Operations Centre.
SPL	Splunk's Search Processing Language.
syscheck	The Wazuh module that performs FIM.
Sysmon	Microsoft Sysinternals system monitor that logs process creation and more.
Who-data	FIM mode that adds the user and process behind a change.

E. Appendix E — References

- Wazuh documentation, File integrity monitoring and Syscheck configuration reference: https://documentation.wazuh.com/
- MITRE ATT&CK, T1565.001 Stored Data Manipulation: https://attack.mitre.org/techniques/T1565/001/
- MITRE ATT&CK, T1112 Modify Registry: https://attack.mitre.org/techniques/T1112/
- Microsoft Sysinternals, Sysmon: https://learn.microsoft.com/sysinternals/downloads/sysmon
- Wazuh SOC Analyst Home Lab — 14-Day Intensive Program, Day 9 brief (Figure 1).

References were not re-checked at the time of writing; verify current URLs and rule IDs against the documentation for your Wazuh version.